ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

SVILUPPO DI UN GIOCO MULTIPLAYER BASATO SU TECNOLOGIE WEBSOCKET E WEBRTC

Tesi di laurea in ingegneria informatica

Relatore

Prof. Paolo Bellavista

Presentata da

Tommaso Ripanti

Sessione ottobre 2025

Anno Accademico 2024/2025

Alla mia famiglia tutta, alla mia compagna e agli amici di una vita. Ai nuovi, con cui ho condiviso tanti momenti di studio.

Indice

Indice 4
Introduzione 6
1 Il gameplay, i mezzi e le alternative 8
1.1 Opzioni di networking 9
1.2 I protocolli utilizzati 13
1.3 Gli strumenti e i framework utilizzati 18
2 Un'Analisi delle tecnologie utilizzate 19
2.1 Gli standard protocollari 19
2.1.1 WebSocket 19
2.1.2 WebRTC 25
2.2 Lato fornitore 36
2.2.1 JVM e JEE 36
2.2.2 Java API For WebSocket 40
2.2.3 Apache Tomcat 43
2.3 Lato utente 44
2.3.1 Cocos Creator 44
2.3.2 JavaScript API For WebSocket 48
2.3.3 JavaScript API For WebRTC 49
3 Il progetto 51
3.1 L'opzione client-server 51
3.1.1 Lato server 56
3.1.2 Lato client 62
3.2 L'opzione peer-to-peer 66
3.2.1 Lato server 69
3.2.2 Lato peer 71
3.3 Le misure a confronto 74
3.3.1 L'opzione client-server 74
3.3.2 L'opzione peer-to-peer 78
3.3.3 Il bilancio 80
Conclusioni 81
Sitografia 83
Ringraziamenti 87

Introduzione

L'oggetto della tesi in questione è un videogame, che vuole porsi da esempio per quella realtà odierna in cui si richiedono tecnologie web sempre più interattive e comunicazioni sempre più ottimizzate. Un contenuto che, ad ogni modo, si fonde alla perfezione con la volontà dell'autore di costruire un suo sistema in autonomia, secondo i suoi interessi e i futuri obiettivi. Oltre a non trascurare il fatto che ha usato quelle tecnologie incontrate nel suo percorso di studio triennale, di cui questo documento ne segna l'epilogo. Non è però solo un videogioco, perché sotto il primo velo si nasconde uno strato di API e protocolli tipici di una rete di calcolatori. Strato il quale, sviluppandosi al di sopra del livello applicativo, si inserisce in quel web di oggi che ben si allontana dalle concezioni di Berners-Lee.

Lasciando da parte l'intento puramente personale di apportare con questo lavoro un contributo alla formazione professionalizzante dell'autore, il carattere sperimentale di questo elaborato propone al lettore un esempio delle possibilità che ci vengono offerte dai browser odierni. Lontani dall'essere unicamente dei clienti HTTP (*HyperText Transfer Protocol*), infatti, e oltre i limiti di un semplice botta e risposta di una architettura *client-server*, per quanto efficiente, i browser di milioni di utenti rappresentano un mezzo comunitario, e non elitario, per informarsi, chattare, fare videochiamate, fruire di contenuti in streaming e, in ultimo ma non meno importante, videogiocare. Sebbene ci sia un'ampia gamma di soluzioni, questo target, a mio dire, è il migliore compromesso per utilizzare e sviluppare un solo sistema per tutti i dispositivi elettronici, di ogni taglia e prestazioni. Per tal ragione, il mio gioco esegue sul browser. E sarebbe bello che il divertimento digitale possa sempre più aprirsi a questi supporti. Questa una speranza tutt'altro che vana, a seguito dei recenti contributi apportati con HTML5 e WebGL, nonostante le difficoltà a mantenere una buona qualità per quei giochi impegnativi che oggi sono pensati, giustamente, per essere nativi e non portatili. Una buona fetta del mercato, però, si può ben adattare ai limiti di queste macchine virtuali, che tendono a incrementare le performance negli anni e a offrire nuove soluzioni interessanti. In primis, la compilazione JIT (*Just In Time*) del motore V8, in sostituzione del semplice, ma pur frenante, interprete e, infine, l'*assembly* intermedio di WASM (WebAssembly) che

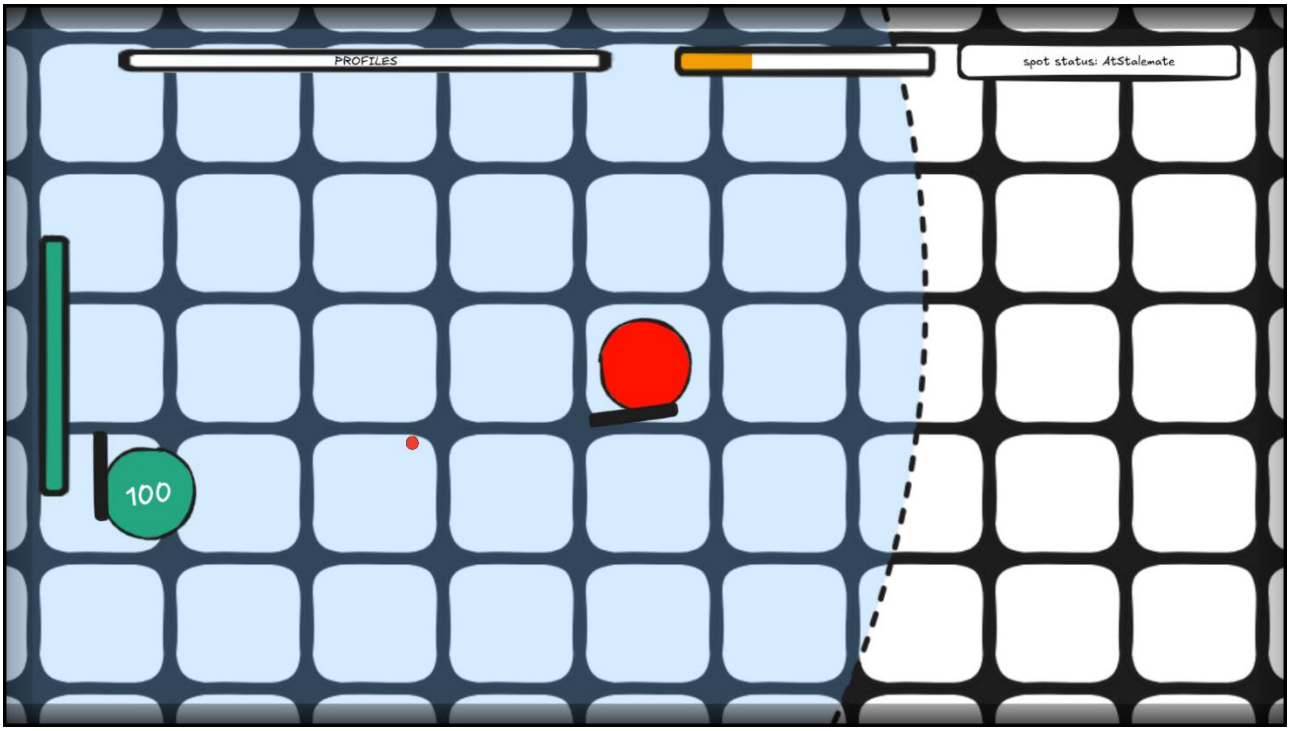
riduce i tempi di traduzione in codice macchina. Per rafforzare l'idea del gioco su browser, non mancano esempi, quali possono essere gli strategici come Forge of Empires, o gli FPS (*First Person Shoother*) come Krunker e molti altri.

Il focus finale va all'aspetto che, più di ogni altro, è stato indagato con questo lavoro e che ha indirizzato l'autore verso nuove conoscenze e competenze. L'aspetto di cui si parla contempla: coniugare gli standard WebSocket e WebRTC (*Web Real-Time Communication*) all'interno di un applicativo a due versioni, una orientata a una architettura *client-server* e l'altra a una architettura *peer-to-peer*; di questi standard spiegare le implementazioni utilizzate e le altre tecnologie che li affiancano; altresì, confrontare i due risultati ottenuti, da cui dedurre vantaggi e svantaggi delle due opzioni di *networking*.

Per concludere questo paragrafo introduttivo, mi appresto a illustrare, in breve, le tematiche dei seguenti capitoli e sottocapitoli. Nel primo capitolo, dunque, si presenteranno i modelli di riferimento, i protocolli di comunicazione, gli strumenti e i *framework* utilizzati così come, con essi, le relative motivazioni che hanno portato l'autore a sceglierli in fase di sviluppo. Nel secondo capitolo, saranno analizzati nel dettaglio gli standard dei protocolli, le tecnologie adoperate lato server e quelle lato client. Infine, un terzo e ultimo capitolo racconterà il progetto, incentrandosi sulle difficoltà incontrate, inquadrando i punti critici con l'aiuto di *snippet* di codice e mostrando i responsi delle misure effettuate, in modo da fare un successivo confronto tra le due modalità.

1 Il gameplay, i mezzi e le alternative

A prescindere dalle ragioni che giustificano il tema scelto e i vari argomenti che ne emergeranno, il filo conduttore rimane il gioco con le sue regole. Dunque, occorre descriverne le dinamiche. Nella pratica, il *gameplay* gira intorno a una meta, cioè assediare un avamposto con l'aiuto dei propri compagni di squadra, fintantoché non ne si completi la conquista. Questo lo si può fare permanendo nella zona per almeno dieci secondi e difendendola da probabili incursioni dei giocatori avversari. In sostanza, squadra rossa e squadra verde combattano tra di loro a colpi di arma da fuoco, così da tenere lo *spot under siege*, così come dice il nome del gioco. Alla morte del proprio personaggio, dopo aver subito danni, viene richiesto di attendere cinque secondi, per poi poter rientrare in partita. Si presenta, quindi, come uno sparatutto in 2d dalle figure geometriche, senza alcuna ambizione di tipo grafico e giocabile con soli mouse e tastiera, per questioni di semplicità e tempistiche. Sono quest'ultimi gli stessi motivi per i quali si è deciso di limitare il complesso a un'unica partita globale, a cui è possibile partecipare in ogni momento e fase di gioco.



1. Schermata del gioco "Spot Under Siege"

1.1 Opzioni di networking

Il progetto su cui ho lavorato si articola in due architetture di rete, secondo il modello di interazione scelto. La prima è un'architettura *client-server* che, però, si discosta dal modello più tradizionale, e se ne spiega di seguito il motivo. Il paradigma *client-server* puro prevede una netta separazione dei ruoli, per cui è il cliente ad avere un comportamento proattivo ed è lui, quindi, ad avviare la comunicazione mediante una *request*. Al contrario, il server si limita passivamente a elaborare una *response,* per rendere accessibile, in qualche forma, la risorsa richiesta dall'altro, ossia il client. Questa comunicazione, per di più, prevede uno *strong coupling*[1] tra i soggetti, a differenza di un modello a scambio di messaggi, per le sue proprietà di essere sincrona e bloccante. Precisamente, con sincrona si intende che è prevista sempre una risposta per un primo messaggio, e con ciò può avere senso che sia anche bloccante, cioè che ponga in attesa il primo mittente, fintantoché questo non conosca la risposta. In più, l'interazione è asimmetrica ma anche dinamica, a indicare l'iniziale ignoranza del server rispetto al suo interlocutore, che invece conosce il suo destinatario. Dinamica perché il server prende successivamente consapevolezza dell'altro, così da poter inviare puntualmente la risposta. Il mittente, quindi, conosce i nodi interessati prima di inviare il suo messaggio e, per tal ragione, la comunicazione è detta diretta. Per concludere l'analisi, il dialogo è sempre uno a molti, per dire che il cliente si interfaccia con un unico servente e il servente con molteplici clienti. Dalla serie di proprietà, ci si accorge della sua semplicità ed efficienza nei casi serviti, ad esempio, dal protocollo HTTP, ma di completa inadeguatezza al tipo di interazione necessaria a un videogioco. Infatti, il cambio di stato di un giocatore, per dirne una, va propagato su ogni nodo cliente. Nodi tali che vogliono essere aggiornati sì, ma senza adottare un inefficiente e macchinoso *pull*, come quelli descritti

[1] Con *strong coupling* si intende che i due nodi devono essere presenti simultaneamente per comunicare.

dagli scenari di *polling*[2], *long polling*[3], *forever response*[4] e così via. Occorrono, dunque, dei *push* da parte del server, così da rendere la comunicazione maggiormente *two-way*. Inoltre, per semplificare e velocizzare le meccaniche di *networking* del videogioco, il servitore e i clienti non sono tenuti a rispondere, a meno di eccezioni come nel caso del *ping-pong*. Questo significa che l'interazione deve essere asincrona e, dunque, certamente non bloccante. Apprese le caratteristiche necessarie per la nostra rete, la si può, allora, riconsiderare da una prospettiva più ampia, in cui il flusso dei messaggi non si limita al tragitto cliente-servente ma si spinge oltre, a raggiungere gli altri nodi utenti. Questo nuovo panorama, ci porta a definire il server come un mero *broker*, con il compito di trasmettere gli eventi verificatisi nell'ecosistema di un giocatore agli altri *endpoint*. L'interazione si svela essere una comunicazione molti a molti, con la possibilità di inviare, in modalità *multicast*[5] *e broadcast*[6], a tutti coloro interessati a ricevere notifiche, cioè chiunque altro si trovi in partita. Il mittente, inoltre, non conosce i suoi interlocutori ma solo un intermediario. Ecco che, quindi, la prima opzione di *networking,* pur chiamandola *client-server*, si scopre avere molto più in comune con un modello a scambio di messaggi, o, in maniera più appropriata, con un modello *event-driven*.

La seconda modalità, invece, stravolge la tipica struttura di rete più volte riproposta durante questo percorso triennale, cioè tanti nodi client e un solo server, o magari più di un server al fine di distribuire il carico, oppure aumentare la ridondanza per essere robusti ai guasti. Infatti, nell'architettura *peer-to-peer* pura, una macchina propriamente server non sussiste, e ogni nodo paritario è in grado di vestirsi sia da servente che da cliente. In

---

[2] Il *polling* consiste in un client che fa molteplici richieste periodiche per ottenere aggiornamenti dal server.

[3] Il *long polling* consiste in un client che fa molteplici richieste consecutive. Il server, però, non risponde finché non ha novità da trasmettere. Da precisare che non viene inviata una nuova richiesta se non arriva risposta alla precedente.

[4] Il *forever response* consiste in un client che fa un'unica richiesta e in un server che non termina mai di rispondere. Invece di inviare una risposta completa, il server manda solamente *chunk* di questa.

[5] Il *multicast* è una modalità di trasmissione in cui si inviano dati ad un gruppo specifico di destinatari.

[6] Il *broadcast* è una modalità alternativa al *multicast* in cui si inviano dati a ogni utente della rete.

particolare, i *peer* formano una *overlay network*, generalmente posta al di sopra della rete internet, e di questa ne è una sottorete, che mappa e indicizza i nodi. Nello specifico, questa rete può essere strutturata o non strutturata. Nel primo caso, la rete viene costruita in modo tale da garantire una ricerca efficiente di una risorsa o di un altro nodo. Quindi le *route* verso una risorsa sono predefinite. Per farlo, si sfruttano delle mappe chiave-valore, le *distributed hash table*, in cui la chiave è un codice *hash* che identifica una singola risorsa e il nodo che la conserva. Per evitare un singolo punto di fallimento, le risorse nella rete vengono replicate su più nodi, e le richieste per una risorsa vengono distribuite con un meccanismo di *load balance*[7], per evitare *bottleneck*. In realtà, è indispensabile qui la presenza di entità stabili che maggiormente si avvicinano al concetto di server. Si parla di *bootstrap node*, cioè dei *peer* noti che permettono ai nuovi di iniziare il *peer discovery* e popolare la loro mappa. Nel secondo caso, invece, la rete si modella casualmente, permettendo ai nodi, senza conoscenze del vicinato, di instradare i messaggi senza fisse regole ma in modo casuale. Questo approccio si rivela più semplice da implementare, sebbene le ricerche siano di minor qualità, del tipo *query flooding*, cioè mediante richieste in *broadcast*. Ciò può comportare traffico elevato e, per eliminare possibilità di loop infiniti, si fa uso di parametri di *time to live*. Inoltre, l'uso di *superpeer* o *peer leader,* in cui concentrare i compiti di ricerca e instradamento, migliora le prestazioni. Nel contesto corrente, però, la seconda architettura scelta si conforma meglio a un secondo modello, più ibrido, di comunicazione *peer-to-peer*. Effettivamente, il modello P2P con *discovery server* presenta caratteristiche che rispecchiano il progetto, perciò l'esistenza di un server di intermezzo che faccia da punto di contatto tra *peer*. Questo servitore, detto anche di *signaling*, si rivela essenziale per passare al *peer* appena connesso i riferimenti degli altri o viceversa. Nel caso in questione, peraltro, si fanno largo altri due componenti, importanti per risolvere le problematiche che nascono dalla *network address translation* degli indirizzi privati, come sono quelli delle macchine in gioco. Nel dettaglio, un server STUN (*Session Traversal Utilities for NATs*) consente ai *peer* di conoscere il proprio

[7] Il *load balance* è una tecnica in cui un nodo *dispatcher* distribuisce le richieste su più *endpoint,* così da evitare che uno solo tra questi venga sommerso dal traffico.

indirizzo pubblico, che sarà poi inviato agli altri *peer* e salvato come indirizzo di destinazione a cui spedire i messaggi. L'altro è un server TURN (*Traversal Using Relays around NAT*) che viene in soccorso in caso di scambio diretto di messaggi non riuscito, a causa di NAT o firewall restrittivi. TURN funge da *proxy* e spedisce i pacchetti in nome del *peer* mittente.

La scelta di implementare l'applicativo con entrambe le architetture è giustificata da chiari scopi. In primo luogo, comprendere a pieno gli ostacoli implementativi che sorgono dalla prima e dalla seconda scelta. In secondo luogo, effettuare un confronto tra i due modelli e, forti dei dati sperimentali, individuare punti a favore e difetti di ambedue, in un contesto ben circoscritto, ossia il videogioco con le sue dinamiche.

1.2 I protocolli utilizzati

In questa sezione, si propongono al lettore diversi protocolli di livello applicativo, alternativi a quello WebSocket, al fine di rimarcare le caratteristiche che lo hanno reso ottimale per questo tipo di sistema. Allo stesso modo si procederà per il protocollo WebRTC, adottato nella seconda architettura di rete.

Si sceglie di presentare anzitutto l'HTTP, perché è dalle sue mancanze che nasce quello WebSocket, di cui si parla come fosse un potenziamento del primo. Ne spiegheremo dopo i motivi. Dunque, l'HTTP offre una comunicazione *request-response* affidabile, perché basata sul protocollo TCP (*Transmission Control Protocol*) a livello di trasporto. Il primo anglicismo ci dice che il server HTTP gioca un ruolo passivo e si limita a fornire risposte. Mentre, con l'aggettivo "affidabile" si intende un servizio che garantisce la consegna corretta e ordinata dei pacchetti inviati. Perciò, a meno di *timeout* dovuti a forti congestioni di rete, all'assenza di un percorso su cui instradare e al *crash* del nodo di destinazione, il pacchetto arriva a meta. In più, la politica di riferimento è *at most one*, quindi è al massimo uno, e uno solo, il pacchetto trasferito al livello superiore dello *stack* OSI, nonostante siano state effettuate ritrasmissioni. HTTP si è evoluto nel tempo con più versioni. Nel dettaglio, HTTP/1.0 dedicava un'intera connessione TCP per singolo ciclo *request-response,* richiedendo *overhead* spesso inutili. Con HTTP/1.1 si è rimediato solo in parte. La possibilità di lasciare aperta la connessione migliorava le cose, ma comportava il problema dell'*head-of-line blocking*. In pratica, non si potevano inviare simultaneamente più richieste fintantoché ogni ciclo non fosse terminato con la sua risposta. Questo creava disagi in caso di una risposta in ritardo che, però, si era costretti ad aspettare senza un vero motivo. HTTP/2.0 risolve l'inconveniente e abilita il *multiplexing* a livello applicativo su ciascuna connessione, che ora è possibile condividere e riusare efficacemente. Ad oggi, questo protocollo viene adoperato per una vasta gamma di siti web, dall'e-commerce a siti di notizie, blog, social network, così come per interazioni *backend* tramite API REST[8]. Infine,

[8] Un'API REST (*Representational State Transfer*) è un'interfaccia di programmazione basata su un'architettura con protocollo HTTP e completamente *stateless*. Il server, quindi, non mantiene alcuno stato.

HTTP/3.0 cambia protocollo di trasporto e quello nuovo, QUIC su UDP (*User Datagram Protocol*), elimina lo stesso *head-of-line blocking* a livello di trasporto. Ciò lo rende appropriato per lo *streaming online* e il *voice-over-ip,* in cui la perdita di un pacchetto è davvero trascurabile rispetto all'insieme della trasmissione audio o video.

Per quanto opportuno in molti contesti, realizzare il progetto con HTTP non sarebbe la scelta migliore per l'assenza di *server push.* In realtà, tecniche sofisticate, e già precedentemente citate, ma altamente inefficienti e innaturali, come il *polling* periodico, il *long polling* o il *multiple connections,* permetterebbero aggiornamenti continui dal server. Un'altra strategia, chiamata *streaming* o *forever response,* inverte del tutto la comunicazione, da server a client, ma la rende anche unidirezionale. Da quest'ultima idea ne viene un vero e proprio *upgrade* del protocollo HTTP, cioè il Server-Sent Events (SSE). Ci si riferisce a esso, come nel caso del WebSocket, con il termine potenziamento, in quanto l'*handshake* iniziale tra i due nodi avviene tramite HTTP. SSE fa ancora di più, perché si fonda completamente su HTTP. Di fatti, l'*handshake* prevede una prima richiesta HTTP GET che avvia una risposta infinita da parte del server, tramite l'opzione "transfer encoding: chunked" dell'*header* HTTP. Il server invia aggiornamenti tramite *chunk* di risposta e la connessione termina con un *chunk* di lunghezza nulla. Come parte di una politica di sessione, SSE fornisce un meccanismo semplice per riprendere la comunicazione *server-client* dal punto in cui è stata precedentemente interrotta, per eventi accidentali. In breve, procede con più tentativi di riconnessione, seguendo una strategia *exponential backoff*[9], e appena possibile il cliente invia un messaggio HTTP di sincronizzazione con l'*id* dell'ultimo *chunk,* come parametro dell'*header*. Per riportare un altro esempio di *exponential backoff,* si pensi alle tempistiche di ritrasmissione dei segmenti in TCP, in caso di timeout per mancato *acknowledgement* (RTO[10]). Esempi di uso del protocollo SSE sono i *live feed,* per aggiornamenti sportivi o per rialzi e ribassi delle borse, così come le *real-time dashboard* per log e

[9] L'*exponential backoff* è una strategia per gestire tentativi ripetuti di operazioni fallite. Consiste nell'aumentare il tempo di attesa tra una richiesta e l'altra, riducendo il rischio di sovraccarico del sistema.

[10] RTO sta per Retransmission Timeout.

statistiche. Un'alternativa, ancora, al protocollo WebSocket, e probabilmente la più adeguata a sostituirlo per le sue proprietà, la si ritrova nel protocollo MQTT (*Message Queuing Telemetry Transport*). Quest'ultimo avanza un modello *publish-subscribe* ottimale nel nostro scenario, se non fosse per la sua compatibilità ridotta con i browser odierni, capaci di gestirlo solo attraverso un *escamotage*: MQTT su WebSocket. Se ne comprende da qui, perciò, l'inconvenienza di un livello aggiuntivo. Ad ogni modo, una rapida descrizione del protocollo ci mostra un'interazione fortemente *loose-coupled* in cui spiccano tre entità. La prima entità è rappresentata dal *publisher* o editore che produce eventi; la seconda dal *subscriber* o sottoscrittore che si interessa solo parzialmente o totalmente a tali eventi, suddivisi per argomento. Da precisare che un nodo della rete può ricoprire contemporaneamente ambo i ruoli; La terza entità è un nodo *broker,* che si occupa di smistare le notifiche agli interessati e capace di memorizzarle a sufficienza, per comunicare in modo posticipato la novità a chi non è collegato al momento. I soggetti clienti, dunque, sono fortemente disaccoppiati, sia perché non hanno bisogno di conoscersi direttamente per interloquire, sia perché non sono neppure obbligati a una presenza simultanea. Per la sua leggerezza ed efficienza, dovuta alle intestazioni dei messaggi notevolmente ridotte, il protocollo viene utilizzato per reti di IoT (*Internet of Things*). Infine, alla pari di MQTT, WebSocket garantisce una comunicazione bidirezionale ed efficiente, in termini di consumo di banda, tra client e server, e lo fa tramite un'unica e persistente connessione TCP. Nel complesso, WebSocket permette di creare un ecosistema, a scambio di messaggi, di tanti giocatori che sfruttano il server come broker che, nondimeno, è capace di mantenere un proprio stato e, eventualmente, essere anche il primo mittente verso gli altri.

Per capire al meglio i vantaggi di WebRTC, si propone un paragone con altri tre protocolli per comunicazioni *peer-to-peer,* attualmente combinati assieme per servizi di *VoIP* (*Voice over IP*) e in generale per lo *streaming* audio e/o video. Innanzitutto, il protocollo SIP (*Session Initiation Protocol*) si dedica alla fase di *signaling,* di inizio e di fine sessione. Precisamente, il nodo chiamante manda un messaggio di tipo *invite* al server SIP, con informazioni quali indirizzo IP e porta, per la comunicazione RTP (*Real-time Transport*

Protocol), oltre ai codec che si possono usare per la codifica della traccia. Il tutto è strutturato in un formato testuale chiamato SDP (*Session Description Protocol*). Il messaggio viene quindi rediretto verso il destinatario finale, che il server conosce previa sua registrazione tramite un identificativo SIP, e la negoziazione continua con i messaggi successivi, cioè un *ringing,* un *ok* e un *ack.* Da questo momento in poi, lo *stream* multimediale viene trasmesso tramite protocollo RTP. Quest'ultimo privilegia l'efficienza di UDP all'affidabilità di TCP, eccessiva per l'obiettivo del protocollo. Tuttavia, l'*header* di RTP aggiunge dei campi del genere *sequence number* e *timestamp,* pensati l'uno per riordinare i pacchetti a destinazione e l'altro per sincronizzare le molteplici tracce trasmesse al momento del *playback*. Un altro campo di rilievo è il *payload type,* che indica il codec di codifica del contenuto. Relativamente al codec, questo è possibile modificarlo dinamicamente durante lo *streaming,* per adeguarsi alle condizioni di rete. Ebbene, i *feedback* su *delay, jitter* e uso di banda, su cui viene calibrato il codec da utilizzare, sono scambiati attraverso un altro protocollo di supporto, ovverosia RTCP (*Real-time Transport Control Protocol*). Come avviene per WebRTC, nella situazione in cui i *peer* si posizionano all'interno di una LAN (*Local Area Network*) con NAT (*Network Address Translation*), si sfrutta un protocollo comune per la ricerca della via di comunicazione a più bassa latenza tra i due nodi. Mentre ICE (*Interactive Connectivity Establishment*) è nativamente integrato in WebRTC, lo sviluppatore è tenuto ad accorparlo manualmente se utilizza il protocollo SIP. La strategia dietro a ICE prevede l'enumerazione dei possibili candidati per l'interazione, da parte di entrambi i *peer*, i quali si preoccupano di scambiarli attraverso messaggi SIP in formato SDP. Nel complesso, il connubio SIP, RTP e RTCP non si allontana molto dalle esigenze di *networking* dell'applicativo finale, bensì si disallinea dai semplici contenuti testuali da trasmettere, effettivamente, nel videogioco *target.* Tanto è che WebRTC, se si fosse trattato di tracce audio o video, al contrario di quel che è stato nel progetto, si sarebbe basato ugualmente sullo stesso RTP. D'altra parte, il supporto oramai universale di WebRTC nei browser moderni e l'uso di un protocollo finalizzato a uno scambio di dati più *general purpose,* quale SCTP (*Stream Control Transmission Protocol*), rende il protocollo scelto una base solida per il contesto.

Le digressioni svolte nei paragrafi sovrastanti ribadiscono la convenienza delle decisioni prese. In particolare, si vuole sottolineare la compatibilità di ambedue i protocolli con i browser odierni e le garanzie offerte dai *layer* di trasporto sottostanti, come ritrasmissioni e ordinamento. Altresì, un altro valido motivo a supporto del primo protocollo, in realtà adottato anche nella seconda architettura per realizzare la fase di *signaling,* è la recente esperienza universitaria che accompagna l'autore nell'uso di quest'ultimo.

1.3 Gli strumenti e i framework utilizzati

Per entrambe le architetture di rete ho scelto di impiegare gli stessi *runtime environment*[11] e linguaggi di sviluppo. Ciò significa che ho adoperato la tecnologia Java Enterprise e i componenti dell'API Java per WebSocket, lato servitore, i cui cicli di vita sono stati gestiti dal *web container* e *web server* compatibile Apache Tomcat. Per lo sviluppo dei client e dei *peer*, dove risiede la *view* del sistema, si è preferito adottare il motore grafico Cocos Creator, le cui *build* possono essere costruite per girare sul browser, sfruttando l'elemento *canvas* di HTML5 e la libreria WebGL per l'elaborazione grafica. Inoltre, si sono adoperate le interfacce di programmazione WebSocket e RTCPeerConnection per comunicare mediante i protocolli relativi. Infine, si precisa che Java è stato il linguaggio scelto per il *backend* e TypeScript quello per il *frontend*.

Le preferenze verso queste tecnologie sono più che mai di carattere personale, nell'accezione delle conoscenze e competenze dell'autore. In effetti, durante il percorso di formazione c'è stato modo e tempo per imparare a padroneggiare la tecnologia Java, tanto in orizzontale quanto in verticale. E per ragione similare, Cocos Creator e TypeScript sono stati i mezzi principali con cui interfacciarsi durante l'esperienza di tirocinio, per non dire che JavaScript e la libreria WebSocket rientrano, comunque, negli argomenti di studio svolti a lezione. Molteplici altre soluzioni tecnologiche si sarebbero potute utilizzare al posto di queste, come Node.js e Socket.IO, Python con libreria websockets lato server, mentre il *framework* PixiJS come alternativa a Cocos Creator. In verità, avrei potuto far uso anche di altri motori che offrono processi di *building* per il web, come Unity e Godot, anche se sarebbe stata meno lineare l'integrazione dei protocolli WebSocket e WebRTC.

---

[11] Con *runtime environment* si intende il più delle volte un ambiente simile ad una macchina virtuale, con la presenza di interpreti che si occupano di eseguire il codice e altri componenti per la gestione della memoria, della *garbage collection* e quella degli errori. Spesso sono offerte con esso delle librerie di utilità.

2 Un'Analisi delle tecnologie utilizzate

In questo capitolo si descrivono le specifiche dei protocolli di rete e delle altre tecnologie utilizzate in fase di progetto.

2.1 Gli standard protocollari

In questa sezione vengono presentati gli standard IETF (*Internet Engineering Task Force*) dei protocolli impiegati. Al contrario, le API relative non rientrano in tale capitolo, ma saranno successivamente discusse assieme alle tecnologie correlate.

2.1.1 WebSocket

Come esposto nel documento RFC6455 (*Request For Comments*[12]), il protocollo WebSocket è stato progettato come un semplice livello aggiuntivo al di sopra del trasporto TCP. Si dice semplice nel senso tale che fornisce davvero poche novità rispetto al TCP, e su ciò si basa il suo successo. Semplice, però, anche nell'accezione di facilità d'uso, in quanto snellisce di molto le operazioni che si è tenuti a svolgere per istaurare una connessione TCP. Alcune delle novità apportate, ad ogni modo, si rivelano indispensabili per conformare il protocollo all'infrastruttura di rete posta a supporto dell'HTTP. Di fatti, WebSocket è ideato per comunicare attraverso le stesse porte del protocollo HTTP, cioè la numero 80 e, nella sua versione sicura, la 443. Questo significa che il protocollo continua a funzionare, nonostante la presenza di *proxy* propri dell'impalcatura web. Dopotutto, l'obiettivo di WebSocket è quello di affiancare a una comunicazione *request-response,* inadatta a scambi pienamente bidirezionali, un canale *full-duplex* che permette la circolazione di messaggi autonomi, tanto da parte del cliente quanto da parte del servente. Si mostra essere, dunque, un potenziamento del classico HTTP, in maniera tale da sostituire i macchinosi algoritmi di *polling*. Riporto ora la serie di attributi che caratterizzano il protocollo. Per primo, l'esistenza di una tecnica di *masking,* che garantisce il funzionamento della comunicazione anche in presenza di nodi intermedi, come *forward* e *reverse*

[12] Una RFC è un documento tecnico pubblicato dalla IETF che può descrivere protocolli o altre specifiche. Sono consultabili pubblicamente online e numerate progressivamente. Da dire che non tutte le RFC diventano degli *standard*.

proxy HTTP. Nello specifico, il *payload* dei messaggi viene cifrato attraverso una semplice operazione di XOR (*Exclusive Or)* tra il contenuto originale e una chiave generata casualmente, mediante un algoritmo pseudocasuale di difficile predizione. Tale cifratura non mira a ottenere confidenzialità e integrità del contenuto, proprietà le quali sono, invece, eventualmente assicurate tramite protocollo TLS[13] (*Transport Layer Security*), nella versione sicura di WebSocket. Tra l'altro, in presenza del *layer* di sicurezza che prevede cifratura simmetrica dei messaggi, il meccanismo diventa obsoleto. Il fatto che la chiave venga spedita in chiaro assieme al contenuto ci dà prova del suo altro intento. Di fatti, lo scopo del *masking* è nascondere ai *proxy* di *caching* il messaggio originale, in modo tale da non interpretare pezzi di esso come richieste HTTP. Questa strategia di offuscamento, per ottenere compatibilità, non viene applicata per i messaggi spediti dal server. Il motivo di tale asimmetria è spiegato dalla certezza che i messaggi inviati dal servitore possono essere mal interpretati come risposte solo se prima è stata fatta una richiesta connessa. Quindi, mascherare solo in direzione *client-to-server* è sufficiente a evitare problemi anche in direzione *server-to-client.* Un'altra importante integrazione è rappresentata dal *framing.* Questo meccanismo riporta la comunicazione a essere suddivisa in pacchetti con confini chiari, che nel protocollo vengono chiamati messaggi. Lo stesso tipo di comunicazione avveniva al livello di rete basato su IP (*Internet Protocol*). La distinzione tra i due livelli sta nell'assenza di limiti nella lunghezza del messaggio. Al contrario, in TCP il dialogo è fatto di *stream* di dati. Nonostante l'unità di scambio sia il messaggio agli occhi del livello applicativo sovrastante, che usa il SAP (*Service access point*) a esso offerto, internamente è adottata una politica di frammentazione, al fine di tollerare una lunghezza imprecisata. La frammentazione segue le regole ora indicate: da un lato, un messaggio non frammentato consiste in un singolo *frame* con bit dell'intestazione FIN a uno e *opcode* diverso da zero. Viceversa, un messaggio frammentato presenta un *frame* di testa con FIN a zero e *opcode*

[13] TLS è un livello dello *stack* protocollare che lavora per la sicurezza della comunicazione, in termini di confidenzialità, integrità e autenticità. Nell'*handshake* iniziale il client verifica il certificato del server con la chiave pubblica del secondo. In una seconda fase viene scambiata la chiave di cifratura simmetrica da utilizzare durante la comunicazione. Per farlo, si usa un algoritmo di cifratura asimmetrica.

diverso da zero, una serie di *frame* con FIN a zero e *opcode* a uno e infine il *frame* di coda con FIN a uno, a rappresentare il termine del messaggio. Da notare che i *frame* detti di controllo non possono essere frammentati ed è possibile interporli, nel *buffer* di invio, a *frame* di un altro messaggio. Questo fatto è un'eccezione alla regola, che impone invece l'impossibilità di alternare *frame* di messaggi diversi. Inoltre, come già ribadito, il protocollo WebSocket garantisce l'ordine dei messaggi in arrivo, così come sono stati spediti. Questo implica, di conseguenza, un ordine dei *frame* secondo uguale strategia. Si è prima menzionato un campo da 4 bit dell'*header* che definisce l'*opcode*, ovvero la categoria del *frame*. Se ne discute a seguire. Oltre ai valori "uno" e "due" che corrispondono alla tipologia "testo" o "binario" in base al formato del *payload*, che nel primo caso è codificato in UTF-8 (*Unicode Transformation Format - 8-bit*), ce ne sono altri che individuano *frame* di controllo. I valori in esadecimale "9" e "A" coincidono con i pacchetti di *ping* e *pong*, mentre il valore "8" con il pacchetto di chiusura della connessione. L'*opcode* "0" specifica che il *frame* è un frammento di mezzo di un messaggio e occorre dunque interpretare il contenuto secondo la tipologia del *frame* di testa. Per concludere la digressione sull'intestazione, altri tre campi RSV1, RSV2, RSV3 sono lasciati vuoti a meno di estensioni, negoziate tra gli interlocutori in fase di *handshake*. In ultimo, tre aree dedicate alla lunghezza in *byte* del *frame*. Sette bit permettono, perciò, di precisare una lunghezza di al massimo centoventicinque *byte,* mentre il valore centoventisei istruisce il nodo a interpretare i due *byte* successivi come un intero senza segno per la lunghezza; Vengono lasciati otto *byte* a seguire, se il valore dei primi sette corrisponde a centoventisette.

```
+-+-+-+-+-------+-+-------------+-------------------------------+
|F|R|R|R| opcode|M| Payload len |    Extended payload length    |
|I|S|S|S|  (4)  |A|     (7)     |             (16/64)           |
|N|V|V|V|       |S|             |   (if payload len==126/127)   |
| |1|2|3|       |K|             |                               |
+-+-+-+-+-------+-+-------------+ - - - - - - - - - - - - - - - +
|     Extended payload length continued, if payload len == 127  |
+ - - - - - - - - - - - - - - - +-------------------------------+
|                               |Masking-key, if MASK set to 1  |
+-------------------------------+-------------------------------+
| Masking-key (continued)       |          Payload Data         |
+-------------------------------- - - - - - - - - - - - - - - - +
:                     Payload Data continued ...                :
+ - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - +
|                     Payload Data continued ...                |
+---------------------------------------------------------------+
```

2. WebSocket *frame*

WebSocket dispone di altre due funzionalità interessanti, che sono possibili grazie all'*handshake* svolto secondo protocollo HTTP. Ebbene, la prima delle quali è una politica CORS (*Cross-Origin Resource Sharing*), per cui il server ha la facoltà di controllare l'origine da cui proviene la richiesta di *upgrade*. In funzione di ciò, stabilire quindi se accettare o meno la nuova connessione. Questo meccanismo necessita, però, del contributo del cliente per essere efficace in termini di sicurezza. Infatti, l'utilità di questo procedimento dipende dal valore del campo "Origin", nell'intestazione della richiesta HTTP di *upgrade*. Il campo dovrebbe indicare l'URL (Uniform Resource Locator) della pagina dalla quale si invia la richiesta, dunque una banale stringa che potrebbe dire del falso. Questo è il caso nel momento in cui il cliente WebSocket non è un *browser* affidabile, ma un qualsiasi altro *host* malevolo. Per garantire la sicurezza, quindi, è importante non avvalersi solamente di questa dinamica. La seconda funzionalità, forse di maggiore utilità, si concentra su altri tre parametri della richiesta dell'*handshake*. Il comune campo "Host" definisce il dominio o l'IP di destinazione. Se basato sul dominio, è possibile che sia una stessa macchina a servire molteplici indirizzi, e questo comportamento è detto *virtual hosting*. Un altro campo "Request-URI" specifica l'*endpoint* del servitore che si preoccupa di gestire l'*upgrade*. Questo vuol dire che un unico *host* è capace di offrire molteplici servizi su WebSocket e tenere aperte, quindi, più connessioni TCP con ugual porta e indirizzo IP di destinazione. Infine, con la stessa coppia della quadrupla TCP è possibile parlare con sintassi differenti. Questo

comportamento è già evidente con la condivisione di porta e indirizzo del servente, da parte di ambedue i tipi di comunicazione tenuti dal cliente: uno con protocollo HTTP e l'altro con WebSocket. Ma c'è di più. Tramite il campo "Sec-WebSocket-Protocol" è possibile interloquire con gerghi diversi al livello sovrastante, sfruttando la medesima porta e il medesimo indirizzo. Per concludere, si illustra prima il procedimento di *handshake* e, dopo, quello di chiusura del protocollo WebSocket. Anzitutto, lato cliente viene effettuata una richiesta di *upgrade,* del genere mostrato in figura.

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

3. Richiesta HTTP di *upgrade*

Si notano vari campi precedentemente descritti o scontati, a meno di quello "Sec-WebSocket-Key" da porre in rilievo. Questo parametro dell'*header* rappresenta una chiave in base-64 che individua in modo univoco tale richiesta, nel contesto della specifica comunicazione HTTP. Viene utilizzato dal cliente come meccanismo di *challenge and response,* così che il server possa provare al cliente di saper dialogare con il protocollo in analisi. Da dire, inoltre, che dall'invio della richiesta la connessione si trova nello stato "connecting". Lato servitore, si procede con l'elaborazione di una risposta come raffigurata nell'immagine.

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

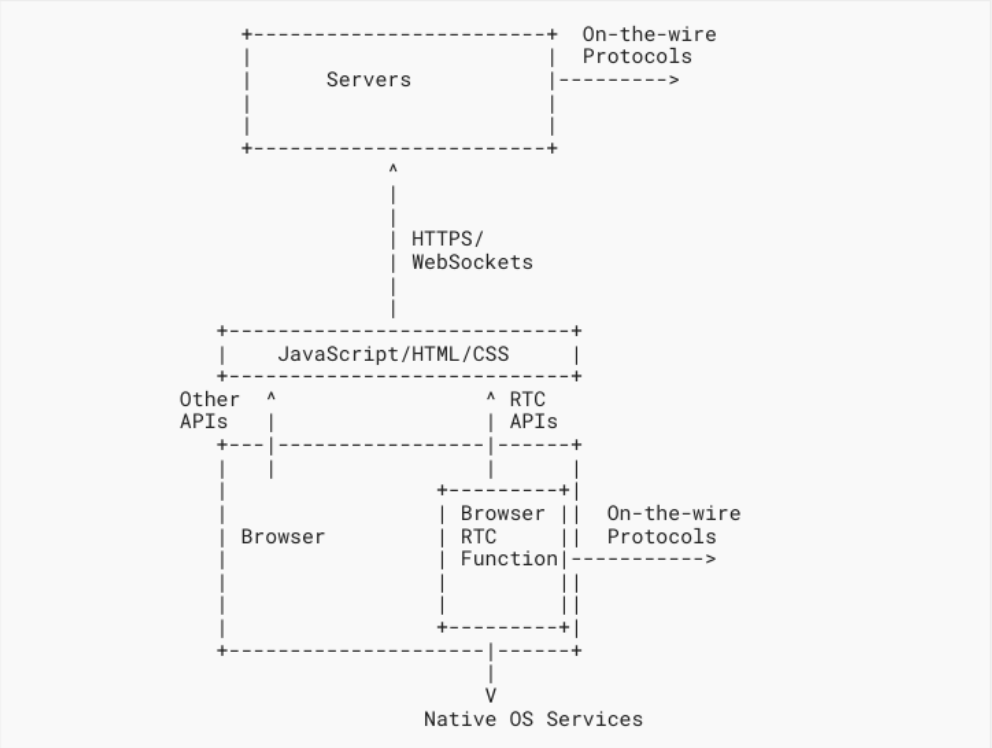
4. Risposta HTTP per *upgrade*

Al campo "Sec-WebSocket-Accept" è assegnato il risultato della trasformazione della chiave presente nella richiesta. Nello specifico, viene effettuata una concatenazione tra chiave e un GUID (*Global Unique Identifier*) che identifica il protocollo WebSocket. Il risultato ottenuto è, quindi, trasformato in *hash* mediante algoritmo SHA-1 (*Secure Hashing*

Alghoritm) e nuovamente codificato in base-64. Al termine, il servitore inserisce il valore nella risposta. Una volta ricevuta dal cliente, questo si preoccuperà di comparare il risultato dei suoi calcoli con quello trovato dal server. Sia nell'eventualità in cui queste stringhe non si uguaglino, sia in caso di codice diverso da "101", la connessione non verrà stabilita e si manterrà semantica e sintassi del protocollo HTTP. In caso contrario, la connessione viene stabilita e lo stato cambia al valore "open". Da questo momento è possibile scambiarsi messaggi WebSocket. Il processo di chiusura è ancora più semplice, perché prevede due messaggi di controllo con *opcode* "8": uno inviato da chi intende avviare la chiusura e l'altro in risposta. Ovviamente, chi invia il *frame* di controllo non dovrà spedire alcun altro messaggio, mentre chi lo riceve scarterà ipotetici messaggi successivi. Nonostante il livello di trasporto sottostante implementi già un *four-way handshake* di chiusura della connessione, si preferisce integrarne comunque un altro, per quelle situazioni che presentano *proxy* intermedi. Effettivamente, questi potrebbero tenere all'oscuro il cliente finale di tale conclusione. A seguito del messaggio di chiusura, si procede con l'interruzione della connessione a livello TCP in modo pulito. Da segnalare che sarebbe opportuno che sia il server ad avviare l'*handshake* TCP e non viceversa. Questo consiglio, presente nel documento RFC, viene citato per ricordare che, in fase di "TIME_WAIT", l'*endpoint* non sarà in grado di istaurare nuove connessioni TCP. Il server, al contrario, supera questa problematica se concorrente.

2.1.2 WebRTC

In base alle specifiche descritte nel documento RFC8825, il protocollo WebRTC raccoglie un insieme di regole per una comunicazione *real-time* tra applicativi *browser-based*. Con l'aggettivo "*real-time*" si intende una comunicazione in cui i dati vengono catturati, trasmessi, ricevuti ed elaborati immediatamente. Inoltre, dai molteplici riferimenti ad altri *standard,* che si richiedono implementati per uno scambio WebRTC, è possibile evincere la natura eterogenea di questo protocollo. Da un lato, il protocollo è tale in quanto offre due soluzioni di interazione: una ideata specificamente per gli *stream* multimediali di tipo audio e video, l'altra pensata per uno scambio di dati di tipo arbitrario. Dall'altro, affinché conforme a una architettura *peer-to-peer,* necessita di vari meccanismi di negoziazione, oltre a utilità per risolvere le problematiche di *endpoint* posti dietro a NAT. L'immagine a seguire mostra i nodi e le interazioni di interesse che possono emergere dall'uso di WebRTC.



5. Modello del browser

In primo luogo, si vuole descrivere la specifica IETF esposta nell'RFC8829, che offre funzionalità a livello di presentazione e sessione. Tale specifica, presentata col nome JSEP (*JavaScript Session Establishment Protocol*), non è assolutamente un protocollo di rete. Bensì, JSEP fornisce una linea guida

per coordinare la fase di negoziazione o *signaling* mediante l'uso di *standard* procedurali differenti. Da una parte si interfaccia con ICE, precisandone le tempistiche di utilizzo. Dall'altra, si preoccupa di codificare i parametri di negoziazione secondo il formato scelto. Non vincola lo sviluppatore a uno specifico formato, nonostante le implementazioni odierne prevedano l'uso di SDP, e lo stesso accade nell'uso del protocollo di rete. Protocollo per mezzo del quale trasmettere i dati di negozio al server di *signaling*. In merito, si riportano nella figura di cui sopra gli esempi di HTTPS e WebSockets. Comunque, per comprendere al meglio, si sceglie di presentare, qui sotto, i contenuti di *offer* e *answer* scambiati tra i *peer* attraverso il servente, così da segnalare i loro indirizzi IP e porte, e decidere i protocolli di comunicazione.

```
v=0
o=- 46117326 2 IN IP4 127.0.0.1
s=-
t=0 0
a=group:BUNDLE data
a=msid-semantic: WMS
m=application 9 DTLS/SCTP 5000
c=IN IP4 0.0.0.0
a=mid:data
a=sctp-port:5000
a=max-message-size:262144
```

6. *Offer or Answer* in formato SDP

Questo è un caso di negozio per stabilire una connessione del tipo *data channel,* cioè tramite SCTP *over* DTLS (*Datagram Transport Layer Security*) *over* ICE *over* UDP. Da qui in avanti, ci si concentra su questa modalità di comunicazione, perché è quella adoperata in fase di progetto. Riprendendo la figura, vale la pena specificare alcuni dei campi di offerta e risposta, come fatto di seguito: "o" definisce l'identificativo di sessione e l'indirizzo Internet del creatore del messaggio; "t" il tempo di inizio e fine della sessione, il quale con un doppio "0" in valore indica una sessione senza limiti; "a=group:BUNDLE data" richiede di usare la tecnica BUNDLE per ottimizzare il trasporto dei flussi di dati, ovvero condividendo la stessa tupla (IP e porta sorgente, IP e porta destinazione, protocollo) tra più comunicazioni; "m=application 9 DTLS/SCTP 5000" stabilisce i protocolli da utilizzare; "a=sctp-port:5000" indica l'identificativo dello *stream* SCTP da usare e "a=max-message-size:262144" la dimensione massima dei pacchetti

SCTP; infine il campo “c=IN IP4 0.0.0.0” è di particolare importanza, dato che il valore “0.0.0.0” specifica che l’indirizzo per la futura comunicazione sarà deciso solo successivamente, grazie al supporto di ICE.

Relativamente a quest’ultimo, ci si rifà al documento RFC8838, che specifica lo *standard* per Trickle ICE. Il procedimento descritto è un’evoluzione della primissima versione e promuove una soluzione più rapida per trovare il canale di comunicazione migliore. La differenza che privilegia il secondo al primo è spiegata a seguire. In sostanza, l’implementazione del primo ICE inviava i candidati rintracciati in blocco assieme all’offerta di *signaling*, solo dopo aver completato la ricerca esaustiva degli stessi. Questo significa che il nodo destinatario dell’offerta avrebbe iniziato la sua ricerca sequenzialmente, in ritardo per il tempo impiegato dal nodo offerente a preparare il suo messaggio. Al contrario, il più recente Trickle ICE permette di separare lo scambio di offerta e risposta da quello dei candidati ICE. Di fatti, un candidato locale viene spedito appena individuato, tramite lo stesso protocollo di rete, cioè quello usato per negoziare. Per precisare, ci sono tre tipologie di candidato che rispecchiano la modalità di comunicazione, che verrà poi utilizzata: candidati *host, server reflexive,* o *relayed*. I primi sono gli indirizzi IP delle macchine *peer,* che vengono direttamente utilizzati se raggiungibili in Internet, oppure in casi in cui entrambi i *peer* si trovino nella stessa LAN; i secondi rappresentano gli indirizzi IP pubblici dei *peer* dalla prospettiva di un nodo esterno alla LAN, e sono ottenuti grazie alla mediazione di server STUN. Queste macchine serventi rispondono alle STUN *request* con l’indirizzo IP e la porta da cui proviene la richiesta. Finché i NAT al confine della rete LAN del *peer* sono di tipo *full cone*[14], è sempre possibile sfruttare questi secondi candidati, per instaurare un dialogo con altri *peer;* i candidati *relayed* rappresentano, invece, gli indirizzi IP dei server TURN, cioè delle macchine che fungono da *proxy* durante l’intera comunicazione tra *peer*. Quest’ultima modalità allunga le latenze e non differisce da un’architettura *client-server*. Secondo le specifiche di Trickle ICE, qualsiasi agente ICE dovrebbe essere in grado di scoprire se l’altro *peer*

[14] Il *full cone NAT* rappresenta un tipo di NAT permissivo, in quanto qualsiasi *host* esterno può rispondere utilizzando stesso IP e porta di destinazione, una volta che un nodo interno avvia una comunicazione in Internet. Gli altri tipi *restricted, port restricted e symmetric NAT* vincolano maggiormente su chi può rispondere.

supporti la versione evoluta, altrimenti dovrebbe disporre della semantica *half-trickle,* per essere retrocompatibile. L'implementazione supportata da JSEP si aspetta, almeno, di trovare un campo *"trickle",* come vuole lo *standard*, nell'offerta o nella risposta, che assicura il supporto per la seconda versione. Le dinamiche di ICE prevedono che la ricerca dei candidati incominci il prima possibile, appena si percepisce l'avvento di una comunicazione imminente. In JSEP, ciò avviene con la scrittura dell'offerta. Lo stesso accade nell'altro nodo alla scrittura della risposta. Ogni agente ICE, allora, crea la sua *checklist* e la pone nello stato di *running* al fine di popolarla alla formazione di nuove coppie di candidati, su cui effettuare i *connectivity check*. Se non viene trovata una coppia valida, una volta che tutti i candidati sono stati individuati, allora la *checklist* va in stato *Failed,* altrimenti passa in stato *Completed*. Alla comparsa di un nuovo candidato, si decide se scartarlo o meno in base alla sua ridondanza nel contesto. Quest'ultima si verifica nel caso in cui il candidato abbia IP, porta, tipo e base di un altro già identificato e se la connettività non risulti migliorata rispetto a quella offerta dai precedenti candidati, per cui i *connectivity check* hanno avuto successo. I candidati funzionali vengono, quindi, condivisi con l'altro agente ICE, che applica una politica di ricezione del tipo *exactly once* e *in-ordered*. Inoltre, è bene che ogni candidato sia correlato a una specifica sessione ICE, così da poterlo accorpare alla giusta *checklist* assieme alla sua controparte remota. Dal lato del mittente, una volta inviato il candidato, questo viene appaiato con gli altri candidati remoti collezionati. Se non ce ne sono, il candidato locale viene inserito solamente nella rispettiva lista. In fase di *pairing*, il candidato locale di tipo *server reflexive* viene sostituito dalla sua base, cioè l'equivalente *host.* Viene poi effettuato un test di ridondanza sulle coppie in stato *frozen* o *waiting,* per eliminare quelle inutili. Gli stati che interessano quest'ultimo processo sono quelli citati, in modo da evitare di rimuovere le coppie per cui si sta attuando un *connectivity check* o per cui il tentativo di connessione ha già svelato un successo o un fallimento. Nel caso in cui la *checklist* contenga il numero massimo di coppie possibili, gli agenti dovrebbero preoccuparsi di cancellare le coppie fallite. Se il problema persiste, si seguono regole di priorità. Lo stesso metodo è applicato in caso di ricezione del candidato remoto. Per rafforzare la descrizione del

meccanismo, si mostra ora il contenuto del messaggio spedito all'altro *peer,* una volta rintracciato localmente un candidato.

```
candidate:842163049 1 udp 1677729535 192.168.1.2 56143 typ srflx raddr 10.0.0.1 rport
56143
```

7. candidato in formato SDP

Nell'immagine di cui sopra emergono diversi valori. Quindi, se ne illustrano i significati a seguire: il primo numero rappresenta la *foundation*, ossia una stringa che identifica i candidati che condividono i medesimi tipo, base, protocollo e STUN/TURN *server,* con lo scopo di supportare i *test* di ridondanza; il secondo è il *component-id* che identifica un flusso di comunicazione; "udp" a indicare il protocollo di trasporto utilizzato; a fianco la priorità del candidato; quindi il suo indirizzo IP e la sua porta; infine "typ srflx" determina il tipo di candidato, mentre "raddr" e "rport" gli indirizzi del server STUN o TURN. Durante la raccolta dei candidati, gli agenti ICE effettuano *connectivity check* secondo un ordine dettato da precise regole, fondate sui valori di *component-id, foundation e priority.* Questi tentativi di connessione consistono in STUN *binding request*, in prestito dal protocollo omonimo, verso l'altro *peer.* Se il controllo va a buon fine, allora la coppia è validata e può essere selezionata come la preferita, nell'ipotesi in cui superi in priorità la coppia migliore al momento. Un caso eccezionale da sottolineare, e di maggior frequenza in Trickle ICE, è individuato nella possibilità di ricevere STUN *binding request* da parte di candidati non segnalati primariamente. In tal caso, il candidato è marcato come *peer reflexive* in tipologia. Questi candidati vengono eventualmente scartati, in caso di ridondanza, all'arrivo di nuove segnalazioni, ma potrebbero comunque far parte della coppia favorita per la comunicazione.

Una volta che almeno una coppia ICE è stata validata e nominata come la favorita, è possibile procedere con l'*handshake* DTLS e quello di SCTP. I canali istaurati non risentiranno assolutamente di una possibile variazione della coppia ICE, grazie alle astrazioni tra i livelli dello *stack* protocollare. Di questo se ne offre una visione molto generale con l'immagine che segue.

```
                  +------+------+------+
                  | DCEP | UTF-8|Binary|
                  |      | Data | Data |
                  +------+------+------+
                  |        SCTP        |
+-----------------------------------+
| STUN | SRTP |        DTLS         |
+-----------------------------------+
|                ICE                |
+-----------------------------------+
| UDP1 | UDP2 | UDP3 | ...          |
+-----------------------------------+
```

8. Diagramma dello *stack* protocollare

Il servizio *data channel* del protocollo WebRTC è l'insieme dei tre blocchi presenti sull'ultimo gradino. Nonostante esso preveda un ulteriore *standard* di *handshake,* cioè DCEP (*Data Channel Establishment Protocol*), in realtà la comunicazione dei dati applicativi continua a basarsi direttamente sul protocollo di rete SCTP. In effetti, il distinguo tra messaggi DCEP e contenuto informativo, in formato UTF-8 o binario, è possibile grazie al campo PPID (*Payload Protocol Identifier*) dell'intestazione dei pacchetti SCTP. Riallacciandosi alla figura sovrastante, è interessante notare come al terzo livello convivano assieme più protocolli. Questo dettaglio richiama l'ottimizzazione BUNDLE citata precedentemente, la quale comportava una condivisione della sessione tra DTLS e SRTP e l'uso di una singola coppia ICE per comunicare (stessa tupla).

In riferimento a SCTP, il protocollo continua a essere *message-oriented* come UDP, ma presenta un proprio concetto di *stream*. Precisamente, uno *stream* SCTP rappresenta una sequenza di pacchetti SCTP con uguale identificativo, che definisce un canale di comunicazione unidirezionale all'interno di una associazione SCTP. Allo stesso modo di TCP, un'associazione SCTP rappresenta generalmente una quadrupla di indirizzi. Tuttavia, la sua funzionalità *multihoming* permette di associare un *endpoint* a un insieme di IP, anche se questa *feature* non è supportata da WebRTC. In più, SCTP fornisce la possibilità di selezionare in modo più fine il grado di affidabilità del protocollo, in termini di ordine dei pacchetti e ritrasmissione. Questo livello di dettaglio è granulare anche perché proprio del singolo pacchetto, e non dell'intero flusso. Come avviene per TCP, propone un controllo di

congestione con oggetti quali la CWND[15] (*Congestion Window*) e tecniche quali *slow start*[16] e *congestion avoidance*[17] in caso di pacchetti non confermati da *acknowledgement*. Non solo questo, SCTP permette messaggi senza limiti, sfruttando la frammentazione del pacchetto in *chunk* che siano al di sotto della MTU (Maximum Transmission Unit). In questo modo si evita l'uso della frammentazione a livello Internet che invece provoca latenze maggiori, per richiesta di elaborazioni aggiuntive nei *router* intermedi, e minor affidabilità. Quest'ultimo deficit in QoS (*Quality of Service*) è principalmente causato dal fatto che occorre reinviare l'intero pacchetto IP nell'eventualità in cui un frammento si perda, visto che non sono previste ritrasmissioni. Riportando l'attenzione sull'MTU, il protocollo SCTP offre un servizio di *path* MTU *discovery,* che regola l'unità di trasmissione massima al posto del protocollo ICMP, su cui si basa, al contrario, TCP. ICMP non può essere adottato in questo contesto poiché non c'è un modo *standard* per legare i suoi pacchetti all'associazione SCTP che ne dovrebbe beneficiare. Nello scenario di WebRTC, si definisce un *data channel* come una coppia di *stream* SCTP con direzioni opposte, identificati da una stringa testuale chiamata *label.* In fase di trasmissione, SCTP può dividere il messaggio applicativo in più *data chunk,* in base alla MTU del momento. Al fine di ottimizzare la trasmissione, il protocollo potrebbe riassemblare più *chunk* assieme, per riempire completamente lo spazio disponibile. Al termine dell'elaborazione, il prodotto che ne esce è un *packet* SCTP formato da un *common header* d'insieme e tanti altri *header* quanti sono i *chunk* nel pacchetto, oltre ai relativi *payload*. In base al tipo di *chunk,* gli *header* correlati si diversificano. Se ne dà quindi un'illustrazione grafica.

---

[15] La CWDN è una variabile mantenuta dal mittente che limita la quantità di dati che possono essere inviati prima di ricevere un *acknowledgement* dal destinatario

[16] Lo *slow start* è un algoritmo che regola la crescita della CWND in modo esponenziale all'inizio della trasmissione, per testare gradualmente la capacità della rete. In sostanza, per ogni ACK ricevuto, la CWND aumenta di un MSS (*Maximum Segment Size*). Quindi, nelle migliori condizioni senza perdite, la CWND aumenta seguendo le potenze di due.

[17] Il *congestion avoidance* è un algoritmo che modera la crescita della CWND. Quando la CWND supera la soglia "ssthresh", si passa da *slow start* a *congestion avoidance*. Per ogni RTT, la CWND aumenta di un MSS, quindi linearmente.

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                        Common Header                          |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                          Chunk #1                             |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                           ...                                 |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                          Chunk #n                             |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

9. *Packet* SCTP

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|     Source Port Number        |     Destination Port Number   |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                      Verification Tag                         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                          Checksum                             |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

10. Common header

Si riportano i significati dei campi di questa intestazione: “Source Port Number” *e* “Destination Port Number” sono autoesplicativi; il “Verification Tag” identifica l’associazione SCTP a cui appartiene il pacchetto; Infine, Il “Checksum” è utilizzato per verificare l’integrità del pacchetto.

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Chunk Type  | Chunk  Flags  |        Chunk Length           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
\                                                               \
/                          Chunk Value                          /
\                                                               \
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

11. Chunk generico

In rapporto al *chunk* SCTP di cui sopra, possiamo dire che: “Chunk Type” definisce la categoria del pezzo tra diversi valori possibili, come per esempio “DATA”, “INIT”, “INIT ACK”, “SACK (*Selective Ack*)”, “HEARTBEAT” e così via; “Chunk Flags” sono una sequenza di 8 bit, con un significato dipendente dal tipo di *chunk*; di seguito il campo “Chunk Length” che specifica la dimensione in byte dell’intero blocco, senza considerare il *padding* finale di zeri, adoperato per rendere il pacchetto multiplo di quattro byte in lunghezza; In ultimo, il “Chunk Value” che contiene altri campi in base al *chunk* di interesse.

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Type = 0    | Reserved|U|B|E|    Length                     |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                             TSN                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|      Stream Identifier S      |   Stream Sequence Number n    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                  Payload Protocol Identifier                  |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
\                                                               \
/                 User Data (seq n of Stream S)                 /
\                                                               \
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

12. *DATA chunk*

Nella cornice mostrata qui sopra, si è deciso di approfondire un *DATA chunk*. Non si dettagliano i campi "Type" e "Length" visti precedentemente. Dunque: relativamente ai bit di *flag* ci limitiamo a "U", "B" ed "E" usati per indicare rispettivamente un *chunk* che non fa parte di una sequenza ordinata, un *chunk* a capo di una serie di frammenti e un *chunk* in coda alla serie. Il parametro "TSN (*Trasmission Sequence Number*)" è un identificativo globale, e non specifico per lo *stream,* utile per la politica di ritrasmissione, quindi riproposto nei *SACK chunk*; lo "Stream Identifier" che collega il blocco dati allo *stream* a cui appartiene; lo "Stream Sequence Number" sfruttato per riordinare i *chunk* all'arrivo; Infine, il "PPID" di cui già si è trattato e lo "User Data" per l'informazione di livello applicativo.

Nella versione utilizzata in WebRTC, il protocollo SCTP sfrutta l'estensione di *interleaving,* che permette di alternare l'invio di *chunk* di *stream* diversi. In tal maniera, il monopolio di banda da parte di un ipotetico grande messaggio, relativo a uno specifico *stream,* viene mitigato. Inoltre, se uno *stream* è in attesa di un *acknowledgement* per un messaggio spedito in precedenza, è sempre possibile inviare messaggi appartenenti ad altri *stream* nel frattempo. Questo concetto è lo stesso del già citato *head-of-line blocking.*

Ci si concentra ora sul protocollo DCEP, per dettagliare il processo di avvio della comunicazione. DCEP è pensato per offrire una semplice modalità di *handshake* con l'uso di due soli messaggi, cioè un *data channel open* e un *data channel ack.* Anche esso si fonda su SCTP e, anzi, adopera gli stessi *stream* utilizzati per il successivo scambio di messaggi di livello applicativo. Per tale motivo, l'*handshake* DCEP è detto *in-band*. Dunque, per differenziare i pacchetti di *handshake* con quelli dal contenuto informativo, ci si avvale

del PPID dell'intestazione SCTP, come prima citato. Nello stadio iniziale, è rilevante capire come vengano scelti gli *stream* attraverso cui dialogare. Nel caso in questione, in cui SCTP si pone al di sopra del protocollo per la sicurezza DTLS, la decisione viene così presa: chi interpreta il ruolo di cliente DTLS fa uso di *stream* con identificativi pari, mentre il nodo con il ruolo di servente adopera quelli con identificativo dispari. Una volta stabilito ciò, viene spedito un messaggio di apertura così come presentato qui sotto.

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|  Message Type |  Channel Type |            Priority           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Reliability Parameter                      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|         Label Length          |       Protocol Length         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
\                                                               /
|                             Label                             |
/                                                               \
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
\                                                               /
|                           Protocol                            |
/                                                               \
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

13. Messaggio di *data channel open*

A seguire, si precisano i significati dei campi: al "Message Type" è sempre assegnato il valore "3" a simboleggiare un messaggio di *data channel open*; il campo "Channel Type" definisce le caratteristiche che dovrà avere il canale, e quindi gli *stream* SCTP. Un codice specifica, perciò, se il canale sarà affidabile, parzialmente affidabile con un massimo numero di ritrasmissioni, oppure con ritrasmissioni fatte entro un timeout. Non solo, segnala anche se il canale sarà ordinato o meno; il campo "Priority", utilizzato dallo *scheduler* SCTP per decidere quali *chunk* inviare, in base agli *stream* di appartenenza; il "Reliability Parameter" usato per trascrivere il numero di ritrasmissioni volute o il *lifespan* per il timeout, secondo il tipo di canale scelto; Per concludere, ritroviamo la lunghezza della stringa di *label*, quella del successivo campo "Protocol" e i rispettivi valori. Il "Protocol" rappresenta esattamente lo stesso concetto di *subprotocol* nel caso visto in WebSocket. Nonostante i messaggi a livello utente possano essere inviati senza attendere una risposta alla richiesta di apertura, il messaggio di *acknowledgement* è decisivo per stabilire il successo o il fallimento dell'operazione. Da specificare che, per quanto nel messaggio di *open* venga proposto altrimenti, i messaggi inviati prima dell'*ack* devono conformarsi al

grado massimo di ordinamento e affidabilità. Dopo la conferma, i pacchetti trasmessi devono rispettare le regole di negoziazione trascritte nel messaggio di *data channel open,* che riguardano lo *stream* complessivo e non un particolare pacchetto SCTP. Di seguito il compatto messaggio di *acknowledgement*, con il solo campo "Message Type" dal valore fisso "2".

```
+-+-+-+-+-+-+-+-
|  Message Type |
+-+-+-+-+-+-+-+
```

14. Messaggio di *data channel acknowledgement*

Per concludere questa sezione, si illustra al lettore il breve processo di chiusura del canale dati di WebRTC. In sostanza, la politica di chiusura prescrive il *reset* dello *stream* in uscita, rispetto al nodo che intende terminare la comunicazione. L'altro *peer* reagirà agendo in ugual modo. Nella sostanza, inizializzare il canale in questo scenario vuol dire porre l'SSN (*Stream Sequence Number*) a zero e notificare questa operazione al livello applicativo.

2.2 Lato fornitore

In questo sottocapitolo si vuole proporre al lettore una visione delle tecnologie che supportano, nel nodo servente, tanto lo sviluppo quanto l'esecuzione dell'applicativo di progetto. La ragione nella scelta del titolo sta nel conformarsi col nome del sottocapitolo successivo. Ad ogni modo, il fornitore è colui che offre un servizio, alla pari del servente, cioè il *provider* che si contrappone all'utente finale.

2.2.1 JVM e JEE

La parte *backend* del sistema di interesse è un programma Java e come tale viene eseguito in una JVM (*Java Virtual Machine*). Come suggerisce lo stesso nome, la JVM è una macchina virtuale, cioè un sistema di livello applicativo che emula la macchina *host* sottostante. Questo significa che offre un ambiente di esecuzione indipendente dal sistema operativo e dall'*hardware* che ospita la VM. Questa propone, al contrario, una sua architettura e un proprio flusso di esecuzione. Tale caratteristica la rende ideale per supportare lo sviluppo di applicativi portabili, ossia programmi che eseguono su qualsiasi *host* fisico in cui una JVM è stata istallata. Il suo progetto segue il principio WORA (*write once, run everywhere*). Dunque, è la sola macchina virtuale a richiedere tante implementazioni differenti quanti sono i *target* che vuole raggiungere, in termini di OS (*Operating System*) e architetture dei processori. L'altro punto a favore dell'uso di una JVM, o di ambienti simili, consiste nell'indipendenza dal linguaggio di programmazione scelto. Di fatti, nonostante Java rimanga il linguaggio più popolare nel contesto, la JVM lavora con un IL (*Intermediate Language*). Questo si traduce nella possibilità di scrivere con qualsiasi sintassi che venga poi compilata nell'IL comprensibile dalla JVM, cioè il Bytecode. Ad oggi, infatti, ci sono altri linguaggi compatibili con la JVM, quali Kotlin, Scala e Groovy. Da questa prima introduzione, si illustrano ora i componenti che rendono la JVM un *runtime environment*. Innanzitutto, si espongono le diverse memorie utilizzate dalla macchina virtuale, quali la *method area*, la *heap*, gli *stack*, i registri PC (*Program Counter*) e gli *stack* per i metodi nativi. In sequenza, nella *method area* vengono salvati i metadati delle classi e dei metodi, come i nomi, il tipo dei campi, e il codice dei metodi. Inoltre, qui è presente anche la *virtual*

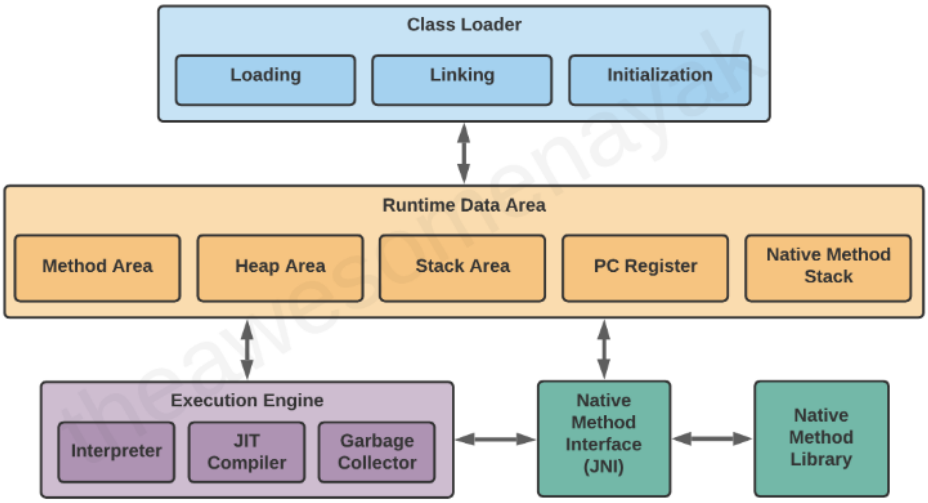
method table, per concretizzare la tecnica di *late binding*[18], essenziale per il polimorfismo[19] della programmazione *object-oriented*. Il PC punta in questa area per leggere le istruzioni; i controlli di tipo sono effettuati verificando i metadati qui presenti e le API per la *reflection* utilizzano queste informazioni per la metaprogrammazione. Nella *heap,* vengono posti gli oggetti, quindi tanto le classi reificate quanto le istanze di esse. Perciò, le variabili di tipo *reference* puntano a questa area di memoria. Comunque, da precisare che queste due aree di memoria sono zone condivise della JVM, ovverosia ogni processo può averne l'accesso. Non accade così per gli *stack* e i registri PC, ognuno associato a un solo *thread,* l'unico il quale può accedervi. Ciascuno *stack* è composto da tanti *frame* quanti sono i metodi invocati e ancora non terminati. Ogni *frame* si tripartisce in *local variables*, *operand stack* e *frame data*: la prima zona è un *array* con i valori (che possono essere anche indirizzi) delle variabili locali, come dice lo stesso nome, ma anche dei parametri in ingresso e del puntatore *this*; la seconda è una struttura LIFO (*Last In First Out*) a supporto delle operazioni intermedie, dove inserire operandi e operatori; l'ultima zona mantiene il riferimento ai metadati della classe nella *method area,* l'indirizzo di ritorno e l'*exception handler table*. Questa tabella mappa il giusto *catch block* da eseguire con l'eccezione da gestire. Relativamente al registro PC, questo viene utilizzato per mantenere il riferimento all'istruzione corrente da eseguire. Infine, gli *stack* per i metodi nativi sono pensati come aree di memoria per l'esecuzione di funzioni scritte in altri linguaggi, come C e C++. Un altro importante componente della JVM è il *class loader*, di cui ne esistono tre tipologie chiave: il *Bootstrap class loader* carica i *package* nativi del JRE (*Java Runtime Environment*), cioè tutti quelli nella *directory* "JAVA_HOME/jre/lib"; si ha poi l'*Extension class loader*, sottoclasse del primo, che si occupa di caricare le estensioni alle librerie *standard*, quindi quelle nel direttorio "JAVA_HOME/jre/lib/ext"; per concludere, l'*Application class loader* che carica i .class presenti nel

---

[18] Il *late binding* è il concetto per cui si decide a *runtime* quale metodo eseguire in risposta ad una chiamata. Indispensabile per concretizzare il polimorfismo universale per inclusione, così come chiamato nella classificazione Cardelli-Wegner.

[19] Il polimorfismo può essere inteso in due modi: la capacità della stessa cosa di apparire in forme diverse oppure la capacità di cose diverse di apparire sotto la stessa forma. Per la programmazione *object-oriented,* la classificazione Cardelli-Wegner specifica quattro tipologie di polimorfismo: per inclusione, parametrico, *overloading* e *coercion*.

classpath. Una classe viene caricata al momento del bisogno, in modo dinamico. Per farlo, si richiama il primo *class loader*. Se questo fallisce, in quanto non trova nella sua *directory* la classe identificata dal suo *fully qualified name*, allora demanda il compito alla sua sottoclasse. Una volta che la classe è caricata in memoria, si passa alla fase di *linking,* suddivisa in *verification*, *preparation* e *resolution*. Nel primo stadio, ci si preoccupa di verificare la correttezza della struttura del .class; nel secondo stadio, si alloca la memoria per i campi statici, cioè le variabili di classe, che vengono poi inizializzati con i valori di *default;* nel terzo stadio, si risolvono i simboli, sostituendoli con riferimenti diretti alle altre classi. Dopo il *linking*, si procede con l'*initialization*. In questa fase vengono chiamati i costruttori di classe e assegnati i valori alle variabili statiche, come prescritto da codice. Considerato tutto, manca ancora una fetta della macchina virtuale: l'interprete. Infatti, il Bytecode dei .class viene generalmente interpretato. A differenza di un compilatore, l'interprete traduce la singola istruzione nel corrispondente codice macchina e la inoltra direttamente al processore. Questo viene fatto riga per riga. In alcuni casi, però, la JVM ottimizza il procedimento sfruttando un JIT *compiler*. Questo tipo di compilatore viene usato in scenari che presentano codice ripetuto, come nei cicli. A seguire, un'immagine che illustra i componenti della JVM.



15. Diagramma dei componenti della JVM

In questo contesto, si precisa che è stato utilizzato il JDK (*Java Development Kit*) offerto dalla JEE (*Java Enterprise Edition*). Il pacchetto mette a

disposizione una serie di librerie aggiuntive per il mondo web. Questo è il caso di componenti come Servlet, JSP (*Java Server Pages*), JSF (*Java Server Faces*), EJB (*Enterprise Java Bean*), che facilitano lo sviluppatore in compiti riguardanti l'interfaccia con il cliente mediante protocolli di rete, la validazione degli input, l'interfaccia con il database, la gestione della sicurezza e la gestione delle transazioni. In questa *suite* rientra anche l'API per WebSocket.

2.2.2 Java API For WebSocket

Una API fornisce una serie di interfacce per interagire con lo strato nascosto che implementa un servizio. Questo concetto applicato alla *Java API for WebSocket*, formalmente JSR 356 (*Java Specification Request*), si traduce in un insieme di firme di metodi e annotazioni a nostra disposizione, per trasmettere e ricevere i messaggi secondo lo *standard*. Prima di illustrare ulteriormente, si specifica che il *namespace* utilizzato in questo progetto corrisponde a javax.websocket, e non al più recente jakarta.websocket. Inoltre, per quanto ci siano delle differenze tra WebSocket *server* e *client*, in questo sottocapitolo ci si concentra solamente sul primo. Accantonando queste precisazioni, per modellare un componente WebSocket, cioè un oggetto di tipo "Endpoint", occorre prima di tutto marcare la classe come tale. Per farlo, ci sono due modalità entrambe ammissibili: una *extension-based* e l'altra *annotation-based*. Nella prima, si specializza la classe "Endpoint" e si attua l'*override*[20] dei metodi. Si sceglie però di descrivere l'API in base alla seconda modalità, così come è stato fatto nel progetto. Essendo all'interno di un modello a contenimento, che sarà spiegato al meglio nella successiva sezione, la classe in questione viene segnalata al *container* come Endpoint tramite l'annotazione "@ServerEndpoint". Peraltro, si può mappare, in modo *inline,* l'URL a cui questo componente deve rispondere, ponendolo tra parentesi immediatamente dopo l'annotazione. All'interno delle stesse è possibile specificare una sottoclasse di "ServerEndpointConfig.Configurator" per personalizzare l'*handshake* iniziale o accedere all'oggetto "HttpSession" che normalmente è di competenza delle Servlet. Tramite le annotazioni "@OnOpen", "@OnMessage", "@OnClose" e "@OnError" è possibile registrare i metodi *handler* opportunamente predisposti per reagire all'avvenire dei rispettivi eventi. Ovviamente, tali metodi devono rispettare la firma prevista, cosicché il *container* possa all'occorrenza effettuare la *dependency injection* per i parametri attesi. Uno dei parametri che viene passato è un oggetto Session, che rappresenta la sessione della comunicazione tra i due *endpoint*. Tale oggetto è utile per impostare variabili di stato della comunicazione e accedervi. Queste azioni si possono intraprendere mediante

[20] Con *override* si intende la ridefinizione nella sottoclasse di un metodo preesistente nella superclasse.

la API “getUserProperties”. In aggiunta, è possibile accedere ai parametri di percorso, definiti attraverso *URI-template,* tramite l’annotazione “@PathParam”. Sempre all’interno dell’annotazione “@ServerEndpoint”, si possono scegliere anche delle classi *encoder* e *decoder,* per serializzare il *payload* del messaggio in un oggetto Java e viceversa. Una classe *encoder* deve implementare una delle interfacce tra “Encoder.Text<T>” ed “Encoder.Binary<T>”. Similmente accade per il *decoder*. Per quel che interessa l’invio di un messaggio, occorre sfruttare l’interfaccia “RemoteEndpoint”, che definisce i metodi “SendText”, “SendBinary” e “SendObject”. L’ultimo dei quali è possibile usarlo solo se definito un *encoder*. Per recuperare un oggetto che implementa tale interfaccia, si adoperano i metodi “getBasicRemote” o “getAsyncRemote” dell’oggetto “Session”. La differenza tra i due sta nel tipo di “RemoteEndpoint” che viene restituito. Nel primo tipo, una *send* invocata si comporta in modo sincrono con i livelli inferiori dello *stack* di rete. Questo vuol dire che il metodo non ritorna fintantoché il messaggio non sia stato effettivamente inviato nella sua completezza. Nel secondo, in modo asincrono e, quindi, ritornando subito dalla chiamata a funzione. Si mostra qui sotto un esempio di implementazione di un *endpoint* WebSocket.

```
@ServerEndpoint("/chat")
public class ChatEndpoint {

    @OnOpen
    public void onOpen(Session session) {
        System.out.println("Connessione aperta: " + session.getId());

        session.getAsyncRemote().sendText("Benvenuto! La tua sessione è: " + session.getId());
    }

    @OnMessage
    public void onMessage(String message, Session session) {
        System.out.println("Messaggio ricevuto da " + session.getId() + ": " + message);

        session.getAsyncRemote().sendText("Hai detto: " + message);
    }

    @OnClose
    public void onClose(Session session) {
        System.out.println("Connessione chiusa: " + session.getId());
    }

    @OnError
    public void onError(Session session, Throwable throwable) {
        System.err.println("Errore nella sessione " + session.getId() + ": " + throwable.getMessage());
    }
}
```

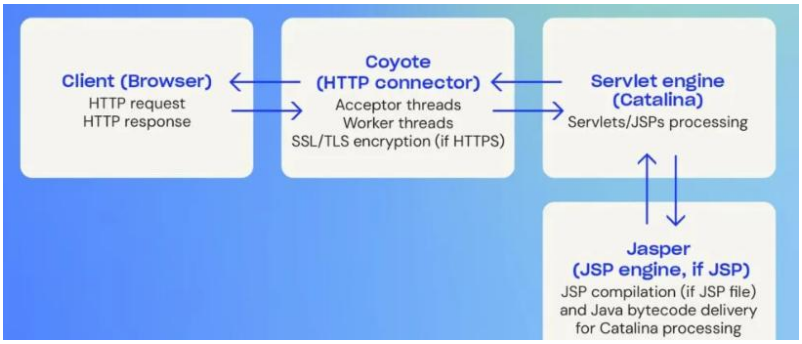
16. Esempio di *endpoint* WebSocket

Per quanto riguarda il ciclo di vita dei componenti, questo viene gestito da un *container,* come già anticipato. Secondo la specifica JSR 356, il *container* deve costruire un’istanza sola dell’Endpoint per cliente e creare un *thread*

per messaggio, così da eseguire gli *handler* predisposti. In questo scenario, dunque, si fa notare come le variabili di istanza hanno uno *scope* di sessione, mentre le variabili di classe hanno uno *scope* di applicazione. Con il termine *scope* si intendono qui i loro tempi di vita e le loro visibilità.

2.2.3 Apache Tomcat

Nell'intreccio di tecnologie finora descritto, Apache Tomcat interpreta il ruolo di *web server* e *container*. Perciò, processa le richieste HTTP e i messaggi WebSocket, così come gestisce il ciclo di vita dei componenti, quali Servlet, *endpoint* WebSocket e altro ancora. Le tre principali strutture presenti nel server Apache sono l'HTTP *connector* Coyote, il *container* Catalina e lo JSP *engine* Jasper. Il primo dei tre rappresenta un semplice *web server* che ascolta sulla porta 80, trasmette le richieste a Catalina e spedisce le risposte elaborate di nuovo verso il cliente. Jasper, invece, si preoccupa di convertire le JSP in Servlet, così che Catalina sia in grado di gestirle. Catalina è considerato un *container* leggero, nel senso che non supporta la tecnologia EJB di JEE. Comunque, questo *container* si preoccupa di creare, inizializzare, eseguire e distruggere tanto le Servlet quanto gli Endpoint WebSocket all'occorrenza. Secondo il *mapping* descritto tramite il file "web.xml", effettua il *dispatch* delle richieste e dei messaggi verso i giusti componenti. Quindi, crea nuovi *thread* al bisogno e invoca i metodi corretti alla ricezione. In questo scenario, Catalina fa anche *dependency injection*, cioè assegna un valore ai parametri e alle variabili dei componenti e, alla necessità, si appresta a costruire nuovi oggetti, come i *managed bean*. In più, gestisce la sessione associata alla comunicazione. Infine, Apache Tomcat garantisce un meccanismo di *hot deployment* e *reload* così da evitare *downtime* del server. In sostanza, una applicazione può essere installata o aggiornata mantenendo il server in esecuzione, senza necessità di riavvio. Per farlo, è sufficiente copiare un archivio .war nel direttorio "webapps/". Sarà poi Tomcat a decomprimerlo e avviare l'applicazione in autonomia. Di seguito lo schema dei componenti in Tomcat.



17. Diagramma dei componenti in Tomcat

2.3 Lato utente

Come è stato fatto per il nodo servente, si propongono ora le principali tecnologie impiegate nel nodo utente. Come promesso al lettore nel sottocapitolo precedente, si spiega brevemente il motivo della terminologia utilizzata nel titolo, che ha peraltro influenzato il nome del precedente. Dunque, si preferisce qui il sostantivo "utente", per fondere assieme i nodi finora chiamati, con maggior precisione, client e *peer*.

2.3.1 Cocos Creator

Allo stesso modo della JVM e del *web container* presenti lato fornitore, anche lato utente sussiste qualcosa di simile. Se da una parte Cocos Creator assomiglia molto più all'impalcatura del *web container,* dall'altra parte il web browser fa le veci della JVM. Sebbene in questa sede tratteremo in modo più approfondito del primo *runtime environment,* si ricordano al lettore i compiti principali svolti da un browser. Di fatti, esso non è solo un mero cliente dei protocolli di rete HTTP e WebSocket, altresì è una vera macchina virtuale capace di interpretare e, in alcuni casi, compilare codice JavaScript. Non si limita a questo, bensì grafica le pagine HTML, traducendole anche in una struttura ad albero, il DOM (*Document Object Model*). Questa struttura semplifica l'interazione degli *script* con gli elementi HTML (*Hypertext Markup Language*), a cui applicare uno stile, tramite fogli CSS (*Cascade Style Sheet*), e per renderli parte di una pagina "attiva". In particolare, con "attiva" si indica una pagina mutevole in base agli input dell'utente, senza bisogno di rielaborazioni lato *server*. Invece, per quel che riguarda Cocos Creator, questo *software* è pensato per essere un ambiente di esecuzione che supporti lo sviluppo di opere quali videogiochi e applicazioni di realtà aumentata e virtuale. Il cuore del motore interessa la gestione del ciclo di vita dei componenti. Questo *loop* viene chiamato a ogni *frame* del gioco, e si preoccupa di invocare i metodi "Start" e "Update" presenti in ciascun nodo. Non solo questo, perché al giusto momento richiama anche i moduli a sostegno del gioco, come l'ActionManager, l'AnimationManager, il CollisionManager, il PhysicsManager e l'InputManager. Ma entriamo nel dettaglio. In una *web*

build[21], come quella utilizzata per il *deployment* del *videogame* di progetto, sono all'incirca questi i passi, illustrati di seguito, che descrivono l'avvio del gioco nel browser e l'inizio del ciclo di vita. La prima pagina processata dal browser, dopo averla richiesta, è quella usuale index.html. In questa, viene descritto il *layout* in cui si trova l'elemento *Canvas* di HTML5, in cui graficare le geometrie del gioco. Nella stessa pagina sono presenti varie direttive che caricano gli *script* necessari per inizializzare l'*engine,* mostrare lo *splashscreen* e allestire la prima scena. La funzione "requestAnimationFrame" dell'oggetto "window" viene sfruttata per ridipingere la tela a ogni *frame*. L'API JavaScript appena citata richiede una *callback* da passare come parametro. Ebbene, questa rappresenta l'elaborazione da compiere prima di ogni *repaint* e deve accettare un valore in ingresso. Tale valore è lo stesso prodotto dalla funzione "now" dell'oggetto "performing", che restituisce il tempo passato dall'apertura della finestra. Nel caso del Cocos *engine,* la *callback* in questione è una funzione definita nell'oggetto *singleton* "game", che richiama però anche il metodo "mainLoop" dell'altro *singleton* "director". Proprio qui scoviamo il nocciolo del motore. Il flusso che si ripete ciclicamente è quindi quello rappresentato nei quattro *snippet* di codice.

```
public mainLoop (now: number): void {
    let dt: number;
    if (EDITOR_NOT_IN_PREVIEW || TEST) {
        dt = now;
    } else {
        dt = (cclegacy.game as Game)._calculateDT(false);
    }
    this.tick(dt);
}
```

```
public tick (dt: number): void {
    if (!this._invalid) {
        this.emit(DirectorEvent.BEGIN_FRAME);
        if (!EDITOR_NOT_IN_PREVIEW) {
            input._frameDispatchEvents();
        }

        // Update
        if (!this._paused) {
            this.emit(DirectorEvent.BEFORE_UPDATE);
            // Call start for new added components
            this._compScheduler.startPhase();
            // Update for components
            this._compScheduler.updatePhase(dt);
```

[21] Con *build* si intende il processo attraverso il quale il codice sorgente di un programma viene trasformato in un pacchetto pronto per essere distribuito. In genere, comprende la fase di compilazione, di *linking* e di *packaging*.

```
        for (let i = 0; i < this._systems.length; ++i) {
            this._systems[i].update(dt);
        }
        // Late update for components
        this._compScheduler.lateUpdatePhase(dt);
        // User can use this event to do things after upda
        this.emit(DirectorEvent.AFTER_UPDATE);
        // Destroy entities that have been removed recentl
        CCObject._deferredDestroy();

        // Post update systems
        for (let i = 0; i < this._systems.length; ++i) {
            this._systems[i].postUpdate(dt);
        }
    }
```

```
        this.emit(DirectorEvent.BEFORE_DRAW);
        uiRendererManager.updateAllDirtyRenderers();
        this._root!.frameMove(dt);
        this.emit(DirectorEvent.AFTER_DRAW);

        Node.resetHasChangedFlags();
        Node.clearNodeArray();
        scalableContainerManager.update(dt);
        this.emit(DirectorEvent.END_FRAME);
        this._totalFrames++;
    }
}
```

18. *Snippet* di codice dei metodi “mainLoop” e “tick”

Nel dettaglio, viene in primis calcolato il *delta time*, cioè il tempo passato tra questo *frame* e il precedente. Vengono poi invocati i metodi “start” dei nodi appena attivati, attraverso l’oggetto “ComponentScheduler”. La stessa entità si occupa di richiamare il metodo “update” per ogni componente di ciascun nodo della scena. Prima di invocare anche le funzioni di “lateUpdate”, vengono aggiornati i sistemi di animazione, fisica e altro ancora, già citati. In questa fase sono processate anche le *callback* impostate dallo sviluppatore tramite le API “schedule” e “scheduleOnce” per *routine* periodiche. Per quanto riguarda i metodi “onLoad” e “onEnable” altrettanto rilevanti, questi sono gestiti da un modulo distribuito su ciascun nodo, cioè il “NodeActivator”. Nell’ultima immagine, compare l’oggetto “root” che mediante il metodo “frameMove” controlla la *rendering pipeline* ai più bassi livelli, interagendo con un’astrazione, il *Graphic Abstraction Layer*, che permette di interfacciarsi con varie librerie grafiche, dipendentemente dal sistema sottostante. Nel nostro caso, con WebGL. Ad ogni modo, si è parlato più volte di nodi e si vuole ora entrare nel merito. Cocos Creator espone un modello *Game Object-Component*. Il *game object*, o nodo, rappresenta un’entità contenitore composta da molteplici componenti che aggiungono nuove funzionalità, in maniera modulare. Sarebbe qui opportuno seguire il principio di singola responsabilità per componente. Si specifica, inoltre, che i nodi sono organizzati in un albero ed elaborazioni a essi collegate vengono effettuate secondo un approccio *deep first* o *cascade*. In pratica, viene prima processato il nodo radice e, subito dopo, il suo primo nodo figlio. Si scende quindi in profondità fino a una foglia dell’albero. Seguendo questa logica, si

completa la gerarchia del primo figlio, privilegiando la profondità. Solo a quel punto si inizia con il secondo figlio e via dicendo. Un altro importante meccanismo è quello degli eventi. Basandosi su un modello *pub/sub*, un componente può registrarsi a un evento propagato da un altro componente tramite il metodo pubblico di *attach,* che lo stesso offre. Più precisamente, è una *callback* a essere legata all'evento. Quando non più interessato, il componente utilizza un metodo di *detach.* Al momento dell'emissione di un evento, le *callback* connesse vengono immediatamente eseguite secondo l'ordine di registrazione.

2.3.2 JavaScript API For WebSocket

L'API JavaScript per WebSocket è intuitiva e di facile utilizzo. Ovviamente si tratta dell'interfaccia offerta dai browser per realizzare un client WebSocket, nonostante la somiglianza con un server realizzato in ambienti come Node.js. Si mostra subito un'immagine da commentare.

```
// Crea una nuova connessione WebSocket
const socket = new WebSocket('ws://localhost:8080');

// Connessione aperta
socket.onopen = function () {
    console.log('Connessione WebSocket aperta');
    socket.send('Ciao server!');
};

// Messaggio ricevuto
socket.onmessage = function (event) {
    console.log('Messaggio ricevuto dal server:', event.data);
};

// Errore
socket.onerror = function (error) {
    console.error('Errore WebSocket:', error);
};

// Connessione chiusa
socket.onclose = function () {
    console.log('Connessione WebSocket chiusa');
};
```

19. Esempio di client WebSocket

Innanzitutto, la prima linea di codice avvia l'*handshake* tramite protocollo HTTP con l'*endpoint* all'URL indicato. Come parametro opzionale, è anche possibile passare un *array* di *subprotocol* che si vuole proporre al server, come previsto dallo *standard*. Controllando il campo "readyState" dell'oggetto WebSocket, si può verificare lo stato della connessione. Comunque, per reagire all'apertura della stessa, o alla sua chiusura, ovvero all'arrivo di nuovi messaggi e al verificarsi di condizioni di errore, si possono adoperare gli *handler* relativi direttamente offerti. Quindi, rispettivamente "onopen", "onclose", "onmessage", "onerror". Un'altra opzione per ascoltare questi eventi è registrare dei propri *handler* tramite la funzione "addEventListener" invocata sull'oggetto WebSocket. Per inviare un messaggio, si utilizza l'API "send" che accetta parametri di diverso tipo: una stringa, un "Blob" oppure un "ArrayBuffer". Se si desidera chiudere la connessione, si può chiamare il metodo "close". Sarebbe bene accertarsi, prima, che non ci siano messaggi ancora da spedire, controllando il valore di "bufferedAmount" della *socket*.

2.3.3 JavaScript API For WebRTC

Questa seconda API è più complicata della precedente, ma si deve comunque tenere conto che gli *snippet,* mostrati qui sotto, prevedono delle parti sempre dedicate all'API WebSocket. Se non fosse per la fase di *signaling* necessaria, infatti, i codici presentati sarebbero più snelli.

```javascript
const signalingSocket = new WebSocket("wss://example.com/signaling");

const peerConnection = new RTCPeerConnection();
const dataChannel = peerConnection.createDataChannel("chat");

dataChannel.onopen = () => {
  console.log("DataChannel aperto!");
  dataChannel.send("Ciao dalla parte offerente!");
};

dataChannel.onmessage = (event) => {
  console.log("Messaggio ricevuto:", event.data);
};

peerConnection.onicecandidate = (event) => {
  if (event.candidate) {
    signalingSocket.send(JSON.stringify({ type: "candidate", candidate: event.candidate }
  }
};

peerConnection.createOffer()
  .then(offer => peerConnection.setLocalDescription(offer))
  .then(() => {
    signalingSocket.send(JSON.stringify({ type: "offer", offer: peerConnection.localDescr
  });

signalingSocket.onmessage = (event) => {
  const message = JSON.parse(event.data);
  if (message.type === "answer") {
    peerConnection.setRemoteDescription(new RTCSessionDescription(message.answer));
  } else if (message.type === "candidate") {
    peerConnection.addIceCandidate(new RTCIceCandidate(message.candidate));
  }
};
```

```javascript
const signalingSocket = new WebSocket("wss://example.com/signaling");

const peerConnection = new RTCPeerConnection();

peerConnection.ondatachannel = (event) => {
  const dataChannel = event.channel;

  dataChannel.onopen = () => {
    console.log("DataChannel aperto!");
    dataChannel.send("Ciao dalla parte rispondente!");
  };

  dataChannel.onmessage = (event) => {
    console.log("Messaggio ricevuto:", event.data);
  };
};

peerConnection.onicecandidate = (event) => {
  if (event.candidate) {
    signalingSocket.send(JSON.stringify({ type: "candidate", candidate: event.candidate }
  }
};

signalingSocket.onmessage = (event) => {
  const message = JSON.parse(event.data);
  if (message.type === "offer") {
    peerConnection.setRemoteDescription(new RTCSessionDescription(message.offer))
      .then(() => peerConnection.createAnswer())
      .then(answer => peerConnection.setLocalDescription(answer))
      .then(() => {
        signalingSocket.send(JSON.stringify({ type: "answer", answer: peerConnection.loca
      });
  } else if (message.type === "candidate") {
    peerConnection.addIceCandidate(new RTCIceCandidate(message.candidate));
  }
};
```

20. Due esempi di peer WebRTC

In primo luogo, si fa notare che le immagini non sono totalmente simmetriche. Ciò è dovuto al fatto che quella di sinistra rappresenta il *peer* che invia l'offerta e riceve una risposta, mentre quella di destra la sua controparte. Innanzitutto, per entrambe viene creata una connessione WebSocket. Dopodiché si alloca memoria per l'oggetto "RTCPeerConnection", che concretizza il concetto di connessione WebRTC. Sebbene sia possibile aprire anche dei *media channel* tramite l'API "addTrack", l'esempio si concentra su l'uso di un *data channel*, materializzato nell'oggetto "DataChannel". Prima di iniziare la comunicazione, occorre negoziarla. Perciò, tramite il metodo "createOffer" si ottiene un oggetto "SessionDescription", che rappresenta l'offerta da proporre. Questo viene salvato localmente con "setLocalDescription". Una volta che la *promise* restituita ha successo, è possibile inviare l'*offer* all'altro *peer,* passando per il server di *signaling*. Chiamare tale funzione vuol dire, anche, avviare il processo ICE. Appena un plausibile candidato viene rintracciato, è invocata la *callback* associata "onicecandidate" che spedisce l'oggetto "RTCIceCandidate" serializzato

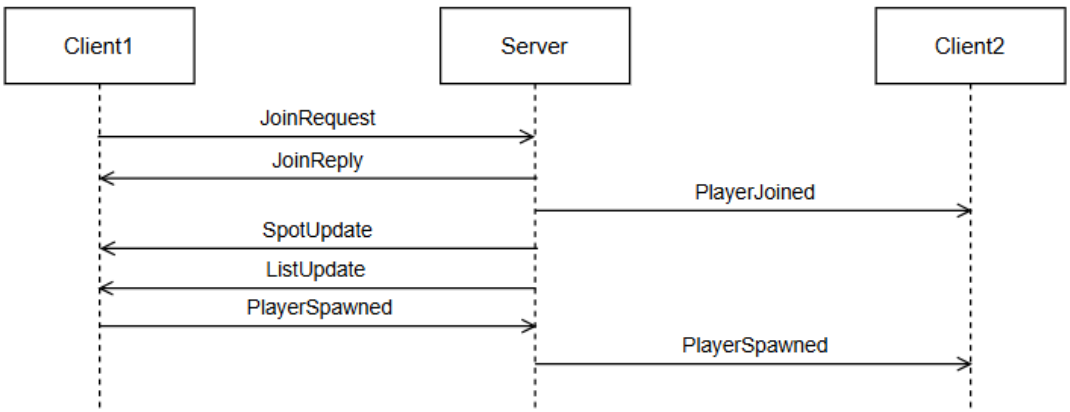
all'altro *peer,* tramite WebSocket. I candidati remoti che vengono ricevuti sono aggiunti alla lista con "addIceCandidate". Mentre, l'offerta o la risposta proveniente dall'altro nodo hanno un loro effetto sulla connessione solo se applicate attraverso "setRemoteDescription". Con l'offerta, il nodo dichiara di voler aprire una connessione *data channel* su SCTP. Se il negozio è andato a buon fine e la connessione SCTP viene istaurata con successo, nel nodo ricevente verrà invocata la funzione "ondatachannel". A seguire, la *callback* "onopen" dopo l'*handshake* DCEP. In più, i messaggi che arrivano a destinazione sono elaborati dall'*handler* "onmessage" dello stesso oggetto "RTCDataChannel", mentre è possibile inviarli tramite l'API "send". Per finire, il metodo "close", invocato sul *data channel,* garantisce un *teardown* pulito della connessione SCTP sottostante, e provoca l'evento relativo nell'altro nodo. D'altra parte, per chiudere definitivamente i rapporti con l'altro *peer* ed eliminare tutte le risorse associate, occorre chiamare il metodo omonimo su "RTCPeerConnection".

3 Il progetto

Con quest'ultimo capitolo, si illustrano le scelte progettuali fatte per la realizzazione del *videogame*. Quindi, si discutono le due opzioni di *networking* del gioco, in merito sia ai flussi dei messaggi scambiati tra i nodi della rete, che alla struttura delle entità coinvolte su ciascuno di essi e alle dipendenze tra queste. Assieme, si presentano i punti critici del sistema e le difficoltà incontrate nell'implementazione.

3.1 L'opzione client-server

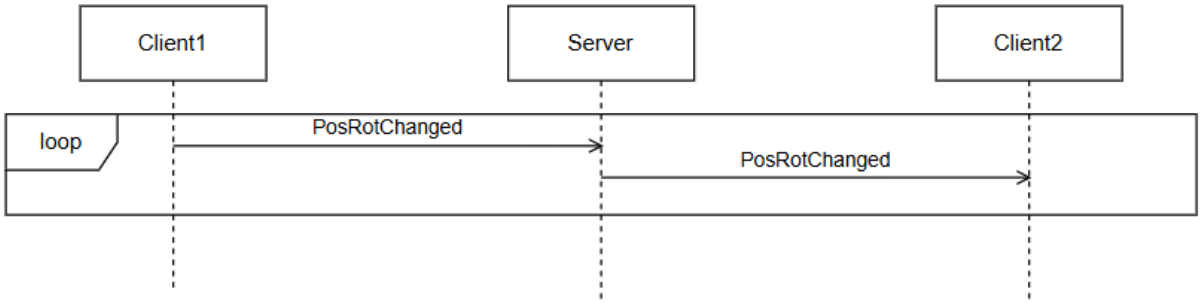
Come avverrà anche per l'opzione *peer-to-peer*, in questa prima sezione si inseriscono una serie di diagrammi di sequenza da commentare. Tali diagrammi offrono al lettore una prospettiva astratta e diretta di buona parte delle conversazioni intrattenute in questa modalità di gioco. Si precisa che, per semplicità, il client numero uno è l'unico a compiere azioni, mentre il client numero due si limita a reagire secondo quanto stabilito. D'altra parte, il servitore si presenta spesso e volentieri come un semplice *relay*. In più, si informa il lettore che l'uso delle frecce con punta vuota simboleggia messaggi che non bloccano l'esecuzione nel nodo mittente, per l'attesa dell'eventuale risposta. Infine, si chiarisce che il carico dei giocatori può, nella realtà, essere di molto maggiore.



21. Diagramma di sequenza: *join* di un nuovo client

Questo primo schema riassume lo scambio di messaggi che avviene nel momento in cui un giocatore decide di entrare in partita. Nell'illustrazione si modella il caso in cui la partita è già in corso, tuttavia può capitare di essere i primi a connettersi. Anzitutto, si spiega lo scenario mostrato e, a seguire,

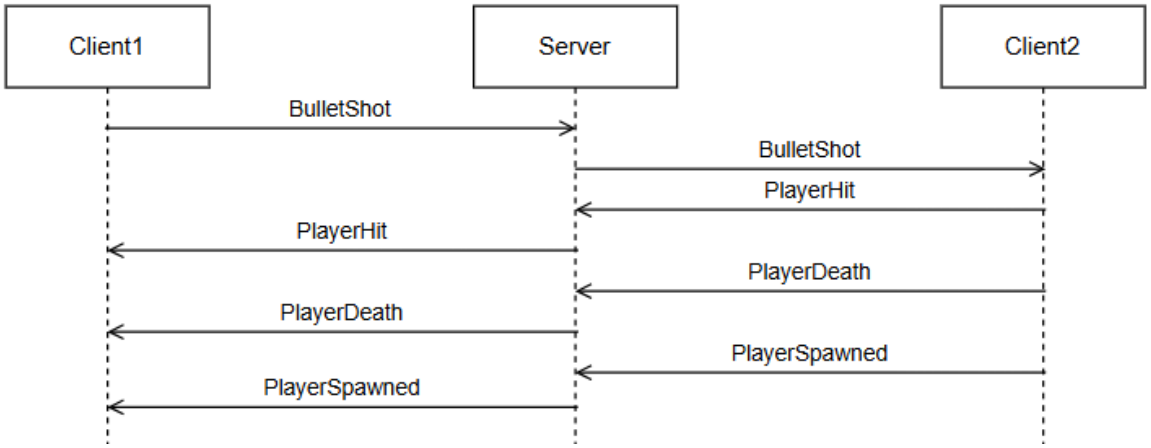
verranno fatte le dovute precisazioni in merito. Dunque, il cliente che intende unirsi al gioco inizia una connessione WebSocket con il servente, lo stesso il quale figura come *endpoint* della connessione HTTP già instaurata. Attraverso quest'ultima è stato possibile inoltrare al richiedente le risorse del gioco, come pagine HTML e JavaScript. Per contro, mediante WebSocket viene inviata la sequenza di messaggi presentata: la "JoinRequest" avvisa banalmente il server della volontà di partecipazione del giocatore, mentre la "JoinReply" trasporta informazioni importanti come l'*id* del player e il team, calcolati centralmente. In particolare, l'identificatore verrà inserito come etichetta di riconoscimento in quasi tutti i successivi messaggi. Il servitore si appresta a propagare l'entrata in gioco del nuovo utente agli altri, tramite un messaggio di "PlayerJoined", con uguale contenuto informativo del precedente. A seguire, invia al giocatore novello uno "SpotUpdate" e un "ListUpdate". Il primo pensato per aggiornare lo stato dello *spot,* locale al cliente, alla situazione di gioco corrente. Similmente il secondo, con la differenza che l'oggetto da adeguare è la lista dei giocatori in partita. Con ciò, il *player* è pronto ad apparire materialmente nell'arena. Così fa e annuncia la sua comparsa con un "PlayerSpawned", insieme alle coordinate del suo nodo nella scena. Questo affinché il server e, soprattutto, gli altri client possano localizzarlo nell'arena, in fase di creazione del nodo debito a rappresentarlo.



22. Diagramma di sequenza: aggiornamento di posizione e rotazione

Durante la partita, ogni giocatore si cura di inviare periodicamente le sue coordinate di posizione e di rotazione locali. Con "locali" si intendono le coordinate rispetto all'origine del nodo padre. Quest'ultime si contrappongono a quelle globali che, invece, sono calcolate rispetto all'origine della scena, così come individuata nel Cocos *engine*. Si sarebbe

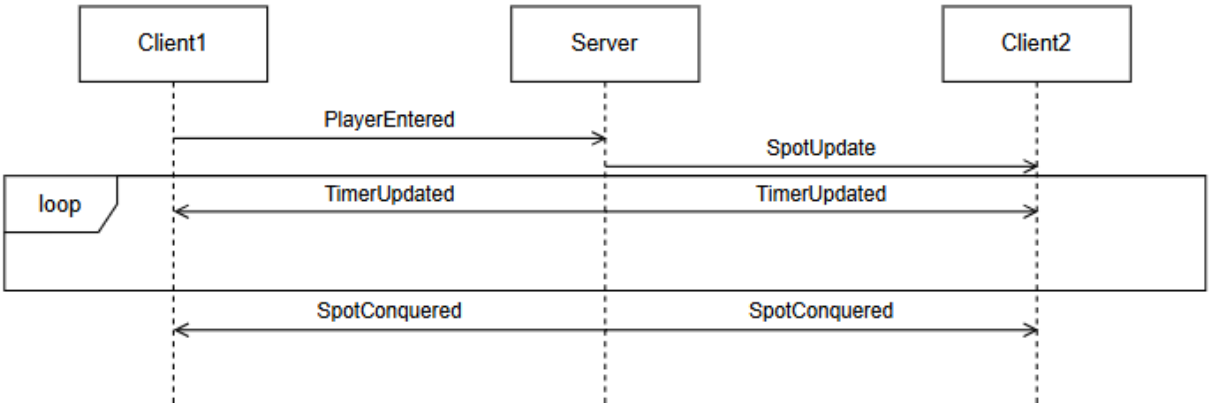
potuto scegliere qualsiasi punto di riferimento per la loro presentazione, in fase di serializzazione e deserializzazione. Bensì, si sono preferite le coordinate locali, in quanto il *node tree* si struttura in ugual modo su ogni cliente. Ciò significa che il nodo padre del personaggio, quello controllato localmente, rispecchia in pieno le coordinate dei corrispondenti nodi genitori negli altri clienti. Detto ciò, il messaggio che trasporta tali informazioni è quello mostrato in figura, sotto il nome di "PosRotChanged". Sicuramente questo è il messaggio che riempie maggiormente il canale di comunicazione, per via delle continue e ravvicinate trasmissioni. Per tale motivo, si rivela importante tarare le tempistiche, al fine di ottenere un buon compromesso tra traffico di rete generato e movimenti senza *lag* all'occhio umano.



23. Diagramma di sequenza: sparo, danno, morte e reincarnazione

Il disegno sovrastante ci mostra la casistica di uno sparo da parte del primo cliente e le sue plausibili conseguenze. Dunque, il giocatore avverte di aver sparato un colpo mediante il messaggio targato "BulletShot". Con esso, si trasmettono le caratteristiche del proiettile, quali: il giocatore che lo ha emesso, la squadra di appartenenza e le coordinate di *spawn*. Dato che la pallottola segue una traiettoria rettilinea lungo il piano dell'arena, e comunque prevedibile dalla cinetica, si preferisce lasciarla al suo corso in ogni macchina, senza uso di aggiornamenti correttivi. Ad ogni modo, il proiettile potrebbe colpire un altro giocatore. Se quest'ultimo si manifesta come un avversario del tiratore scelto, allora il ferito si accinge a spedire un messaggio di "PlayerHit". Unicamente nello scenario in cui il danno d'arma da fuoco provoca la morte, il giocatore caduto segnala il suo decesso tramite un "PlayerDeath". Come raffigurato nell'immagine, però, i personaggi sono

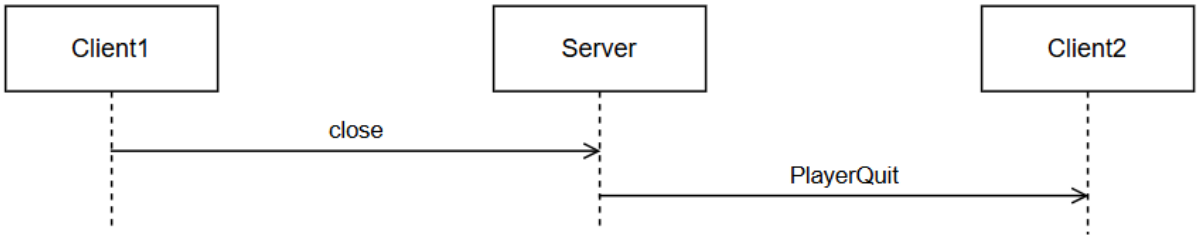
capaci di reincarnarsi allo scadere del tempo necessario per rientrare in partita. Lo fanno tramite un nuovo "PlayerSpawned", con diverse coordinate al loro risveglio, individuate come un punto casuale di un cerchio, con origine e raggio fissi. Questo meccanismo offre all'altra squadra un temporaneo vantaggio numerico.



24. Diagramma di sequenza: assedio dell'avamposto

Questo ulteriore schema è probabilmente il più interessante per la ragione esposta di seguito: mentre gli altri messaggi finora descritti avvisavano tutti di un cambiamento interno al giocatore locale, il "PlayerEntered" riguarda una variazione di stato relativa ad una risorsa posta sul server e, quindi, remota per ciascuno. Tale punto cancella un'importante assunzione su cui ci si è attenuti fino a questo momento. L'assunzione, alquanto vera, consiste nell'affermare che gli avvisi sullo stato di un nodo remoto, ricordando che solo quel nodo può alterare le proprie variabili, arrivano a destinazione secondo l'ordine di invio. Questo è garantito dalle proprietà del protocollo WebSocket e dal comportamento predefinito di Tomcat che prevede un *thread* dedicato per cliente. Quest'ultimo fa sì che i messaggi vengano forzatamente elaborati in sequenza. Nel caso il *thread* sia impegnato alla ricezione di un messaggio, allora quest'ultimo viene accodato in un *buffer* di input. Ciò permette di mantenere l'ordine dei messaggi anche in fase di processamento di questi sul server. Nell'eventualità in cui si fosse scelta la modalità non bloccante NIO (*Non-blocking Input Output)* del *web server*, si sarebbe dovuta implementare in autonomia una coda di ricezione per tale scopo. Se non fosse importante elaborare i messaggi in ordine, ma solo rispedirli in tal modo verso gli altri giocatori, l'altra possibilità sarebbe stata etichettare il messaggio con un *id* sequenziale. Ad ogni modo, le azioni sullo *spot* condiviso

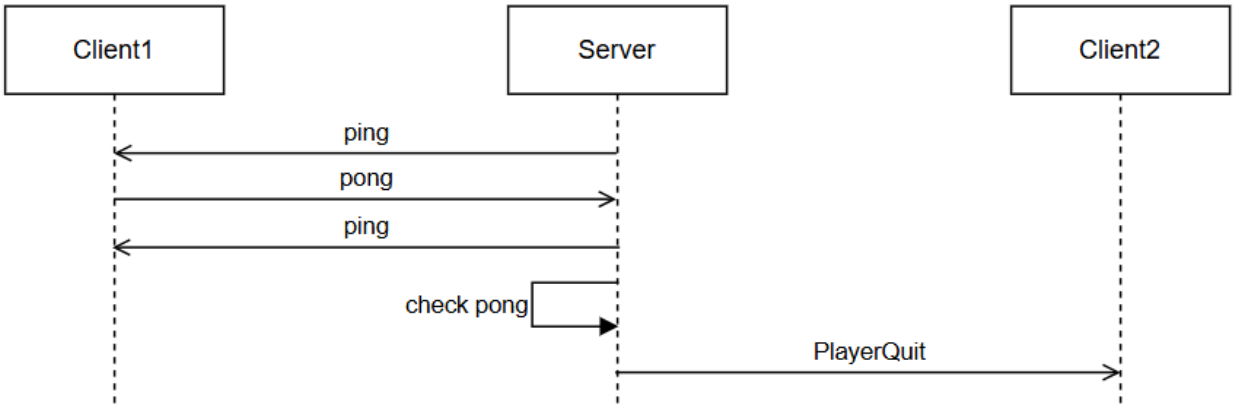
provengono, invece, da clienti differenti. Questo implica che i "PlayerEntered" raggiungano il servitore in modo disordinato rispetto alla loro emissione locale, in base alle diverse latenze di ciascuna connessione. Un buon compromesso, quindi, è aggiungere al messaggio un *timestamp* che segnali il momento dell'invio del messaggio, trascurando, ragionevolmente, le minime asincronie degli orologi di ciascuna macchina. Ribadisco l'avverbio "ragionevolmente", in quanto ogni *host* è obbligatoriamente collegato a Internet e, perciò, il suo orologio è costantemente sincronizzato mediante NTP (*Network Time Protocol*), secondo i meccanismi offerti dal sistema operativo. Ebbene, i "PlayerEntered" vengono salvati centralmente in una coda, secondo il *timestamp* impresso. Se compare un messaggio fuori ordine, il server provvede a ripristinare lo stato corretto dell'avamposto, a meno del *timer* correlato, e ad aggiornare la coda. Il nuovo stato viene divulgato con uno "SpotUpdate". Ad ogni modo, si sceglie di approfondire il meccanismo nella sezione successiva. Per quanto riguarda l'assedio, che si verifica alla presenza di una sola squadra a presiedere l'avamposto, questo viene completato al trascorrere di un certo tempo. In merito, il cronometro che ticchetta è posto sul server e associato allo *spot*. Per ogni secondo, è previsto un messaggio di "TimerUpdated" che aggiorna i nodi remoti. In conclusione, la conquista dell'avamposto è confermata dal server tramite uno "SpotConquered", che porta in grembo il colore della squadra vittoriosa.



25. Diagramma di sequenza: abbandono del client con segnalazione

Nel diagramma in figura, si mostra una delle due modalità per rilevare una disconnessione. Nel caso raffigurato, la volontà di abbandonare la partita e chiudere la comunicazione WebSocket è palesata dal cliente, tramite l'evento "close" dello stesso protocollo. Infatti, nonostante il gioco non preveda nessuna interfaccia grafica per uscire dalla partita in corso, chiudere la *tab* del *browser* in cui è stato caricato il gioco si traduce esattamente in questo. Dato che il *web client* è ancora in esecuzione, si assume lui la responsabilità

di terminare la connessione TCP sottostante. Questo provoca la chiamata del metodo "OnClose" sul servitore, il quale provvede a segnalare l'abbandono avvenuto agli altri clienti, mediante un messaggio di "PlayerQuit". In realtà, questa sequenza di eventi si verifica, generalmente, anche nel caso della chiusura dell'intero *browser*, se questa accade in modo pulito, senza *crash* associati.



26. Diagramma di sequenza: ping, pong e abbandono del client senza segnalazione

In caso di *shutdown* anomali o in presenza di forti congestioni di rete, i messaggi di *ping* e *pong,* previsti dallo stesso protocollo, ci vengono in aiuto. Di fatti, se l'altro *endpoint* non risponde, secondo i tempi stabiliti, a un *ping* del servente, allora è quest'ultimo a troncare i rapporti con l'altro e ad avvisare gli altri dell'accaduto, sempre tramite un "PlayerQuit".

3.1.1 Lato server

In questa sezione, si affrontano gli ostacoli incontrati durante la fase di sviluppo e implementazione nel nodo servitore. Prima di far ciò, si descrive rapidamente la struttura delle classi, così come è stata predisposta. Il complesso ruota intorno alla classe "WebSocketController", che rappresenta *l'endpoint* verso cui instradare i messaggi WebSocket. La sintassi utilizzata è quella presentata nel sottocapitolo "2.2.2 Java API For WebSocket". Al contorno, invece, ci sono una serie di classi che modellano il dominio del sistema. Se ne cita la classe "Match" che rappresenta la partita globale, la classe "Spot" per delineare l'avamposto e quella "Player" a interpretare il ruolo del giocatore. Oltre a ciò, sono presenti altri oggetti di utilità, come l'entità "Message" e la "SessionMessageQueue". Il primo è funzionale per differenziare le azioni, in base al tipo del messaggio WebSocket, che potrebbe

essere uno tra quelli presentati nei diagrammi precedenti. Il secondo è responsabile della spedizione dei messaggi di uno specifico cliente, e a tal fine implementa una coda di questi.

Alla luce di ciò, si discute anzitutto la tematica della concorrenza e le modalità di risoluzione dei conflitti inerenti, nel mondo Java. Prima di illustrare concretamente cosa si è fatto, si sottolinea ancora la natura *multi-threaded* del server e *container* Apache Tomcat, secondo la sua configurazione predefinita. Questa proprietà rende il servente reattivo ad accettare sempre nuove connessioni a livello di trasporto, ed è quindi una soluzione scalabile. Riduce di molto i tempi, che sarebbero, invece, impiegati in uno scenario sequenziale, con code di attesa, per elaborare le richieste, sempre più lunghe al crescere del carico. Se, però, la concorrenza comporta benefici in tal senso, coordinare i *thread* nell'accesso alle risorse non sembra, poi, essere un affare così banale. Con questo panorama, spieghiamo cosa è stato fatto per risolvere potenziali conflitti di contesa delle risorse. In primo luogo, *getter* e *setter* dell'oggetto "Match" e dei suoi componenti sono stati etichettati come *synchronized.* Questo vuol dire che non possono essere effettuate letture e scritture contemporanee, da parte di più *thread,* su quella specifica istanza. In secondo luogo, per tutte le interfacce "Map" si è utilizzata l'implementazione "ConcurrentHashMap". Questa permette un'alta concorrenza, garantendo *thread-safety,* ma senza bloccare l'intera mappa quando si accede ad un elemento di quest'ultima. Infatti, le letture non sono mai bloccanti, mentre le scritture lo sono per la sola coppia chiave-valore di interesse. Inoltre, la lettura non porta mai a una inconsistenza, ma restituisce sempre il valore antecedente o quello successivo a una scrittura. Ancora, le strutture dati derivate sono anch'esse sicure, come nel caso dei "Set" ottenuti con "keySet" o "values", ed eventuali cicli, fatti su di essi, adoperano iteratori *thread-safe*. Per le altre strutture dati, quali le liste ad esempio, non esistono implementazioni equivalenti offerte dalle librerie del JDK. Ho quindi preferito usare l'API alternativa "Collection.synchronizedList", che appone sì il *tag synchronized* su ciascuna operazione effettuata sulla propria lista, ma non offre le garanzie precedenti. Infatti, occorre inoltre racchiudere qualsiasi iterazione sulla struttura all'interno di un blocco *synchronized*, altrimenti si potrebbe incorrere nell'eccezione

“ConcurrentModificationException”. Per concludere l’argomento, è bene dire che, per quanto ovvio, operazioni separate, ma da effettuare assieme per mantenere coerenza, devono inserirsi in opportuni blocchi *synchronized*.

Un’altra questione significativa riguarda la classe “SessionMessageQueue” precedentemente menzionata. Infatti, l’oggetto è stato pensato per colmare le mancanze dell’API “getAsyncRemote” dell’oggetto “Session”. Il metodo restituisce un oggetto con interfaccia RemoteEndpoint che permette l’invio asincrono dei messaggi WebSocket. Questo significa che il *thread* non si blocca in attesa del completamento dell’operazione da parte dei livelli sottostanti. A differenza dei *send* sincroni del “BasicRemoteEndpoint”, le trasmissioni non creano problemi di concorrenza per tentativi di invio simultaneo. Ciò è dovuto al tipo di implementazione che c’è dietro, ossia un *buffer* che accoda i messaggi e serializza gli invii. Tuttavia, il *buffer* presenta dimensioni limitate e, per quanto configurabile in grandezza, si è scelto di risolvere il problema tramite una struttura dati dinamica. Da qui l’adozione della classe “SessionMessageQueue” che incapsula, di fatto, una “ConcurrentLinkedQueue”. Un’istanza della classe viene costruita per ogni cliente e, dunque, associata alla sua sessione una volta aperta la connessione. Nel particolare, ogni qualvolta che si invoca il metodo “send”, tale è il procedimento, come mostrato negli *snippet* di codice sottostanti.

```java
public void send(Message message) {
    queue.add(message);
    trySendNext();
}

private void trySendNext() {
    if (sending.compareAndSet(expectedValue:false, newValue:true)) {
        sendNext();
    }
}
```

```java
private void sendNext() {
    if (!session.isOpen()) return;

    Message message = queue.poll();
    if (message == null) {
        sending.set(newValue:false);
        return;
    }
```

```java
if (sendingNow.compareAndSet(expectedValue:false, newValue:true)) {
    try {
        long [] timestampsCopy = Arrays.copyOf(message.getTimestamps(), message.getTimestamps().length);
        Message msg = new Message(timestampsCopy, message.getType(), message.getValue());
        msg.getTimestamps()[2] = System.currentTimeMillis() - message.getTimestamps()[2];
        session.getAsyncRemote().sendText(gson.toJson(msg), result -> {
            sendingNow.set(newValue:false);
            if (result.isOK()) {
                sendNext();
            } else {
                sending.set(newValue:false);
            }
        });
```

```java
        } catch (Exception e) {
            sendingNow.set(newValue:false);
            sending.set(newValue:false);
            e.printStackTrace();
        }
    } else {
        queue.add(message);
        sending.set(newValue:false);
    }
}
```

27. Snippet di codice da SessionMessageQueue

Come si vede nel primo *snippet*, viene aggiunto in coda il messaggio. Secondo poi, si tenta l’invio del messaggio più vecchio con “sendNext”, a meno che

un altro *thread* non stia già lavorando per svuotare la coda in sequenza. Per verificare ciò, si valuta lo “AtomicBoolean” dal nome “sending”, e lo si imposta a *true* se trovato a *false*. Questo oggetto incapsula un booleano accessibile in modo *thread-safe,* e nel caso blocca le elaborazioni successive da parte di altri *thread*. Con “sendText” viene quindi inviato il messaggio, ottenuto tramite “poll” sull’oggetto “queue”. Questo accade previa verifica dell’altro “AtomicBoolean”, con identificatore “sendingNow”. Questo passaggio attesta che nessun altro *thread* stia inviando un altro messaggio, compreso un *ping*. Infatti, da notare che questa classe gestisce internamente anche una fetta del meccanismo di *ping-pong*. Quest’ultima categoria di messaggi, però, non viene accodata e, in caso di impossibilità di una spedizione istantanea, il *pong* successivo viene condonato. Ritornando al flusso principale, se l’invio del *text* ha successo, allora si prosegue con la trasmissione successiva. Ovvio che, se la *reference* al messaggio fosse nulla, allora verrebbe terminata la *routine*.

Relativamente agli eventi che inficiano lo *spot*, quali i messaggi “PlayerEntered” e “PlayerExited”, si mostra nell’immagine come questi vengono processati e la cui coda riordinata, in caso di bisogno. Si porta come esempio l’*handler* per il primo tipo tra i due.

```
private void onPlayerEntered(Message msg, Session session){
    synchronized(enterExitMessages){
        if(
            enterExitMessages.size() != 0
            && msg.getTimestamps()[0] < enterExitMessages.get(enterExitMessages.size() - 1).getTimestamps()[0]
        ){
            restoreSpotState(msg);
        }else{
            enterExitMessages.add(msg);
            Team team = gson.fromJson(msg.getValue(), Team.class);
            match.getArena().getSpot().getPlayersInsideSpot().put(team, match.getArena().getSpot().getPlayersInsideSpot().get(team) + 1);
            //System.out.println("Player in team" + team + " entered to the spot");
        }

        if(enterExitMessages.size() > 10) enterExitMessages.remove(index:0);
        manageTimer();

        idSessions.forEach((i, s) -> {
            spotUpdate(match.getArena().getSpot(), s);
        });
    }
}
```

28. *Snippet* di codice di "onPlayerEntered" nel “WebSocketController”

Innanzitutto, si sottolinea come l’intera operazione deve essere atomica per quel che riguarda la coda dei messaggi. Effettivamente, è bene che la coda “enterExitMessages”, l’oggetto “Spot” e il timer associato vengano aggiornati tutti al contempo, per consistenza. Continuando col costrutto condizionale, il

messaggio ha effetto immediato solo se temporalmente collocato dopo l'ultimo messaggio trovato in coda. Da chiarire che, anche se si parla di coda, in realtà questa è implementata secondo l'interfaccia "List". In caso contrario, si procede con il ripristino dello stato dell'avamposto e con il riordinamento della coda mediante il metodo privato "restoreSpotState". L'algoritmo del quale sarà illustrato a breve. Nel mentre, in entrambi i casi si continua eliminando i messaggi più vecchi se raggiunta la soglia limite. Questo avviene per evitare di consumare inutilmente memoria per quei messaggi oramai lontani nel tempo e di improbabile utilità. Quindi, viene aggiornato il cronometro in base allo stato corrente dell'avamposto e, infine, quest'ultimo propagato.

```
private void restoreSpotState(Message msg){
    synchronized(spotTimer) {spotTimer.cancel();}
    List<Message> temp = new ArrayList<>();
    int index;
    Message m;
    Team team;
    Spot spot = match.getArena().getSpot();
    synchronized(spot){
        spot = new Spot(spot.getStatus(), spot.getSiegedBy(), spot.getTimer(), spot.getMaxTime(), spot.getArea(), new HashMap<>(spot.getPlayersInsideSpot()));
        synchronized(enterExitMessages){
            for(index = enterExitMessages.size() - 1; index >= 0; index--){
                if(msg.getTimestamps()[0] > enterExitMessages.get(index).getTimestamps()[0]) {
                    temp.add(msg);
                    break;
                }
                m = enterExitMessages.remove(index);

                team = gson.fromJson(m.getValue(), Team.class);
                spot.getPlayersInsideSpot().put(team, spot.getPlayersInsideSpot().get(team) + (m.getType() == MessageType.PlayerEntered ? -1 : 1));
                updateSpotStatus(spot);

                temp.add(m);
            }
        }
        for(index = 0; index < temp.size(); index++){
            m = temp.get(index);
            team = gson.fromJson(msg.getValue(), Team.class);
            spot.getPlayersInsideSpot().put(team, spot.getPlayersInsideSpot().get(team) + (m.getType() == MessageType.PlayerEntered ? 1 : -1));
            updateSpotStatus(spot);

            enterExitMessages.add(temp.get(index));
        }
        spot.setTimer(timer:0);
        match.getArena().setSpot(spot);
    }
}
```

29. *Snippet* di codice di "restoreSpotState" nel "WebSocketController"

Nella funzione "restoreSpotState" è subito evidente come il *thread* che si occupa di incrementare il timer venga terminato, mediante la chiamata a "cancel" dell'oggetto "spotTimer". L'operazione è parte della scelta fatta di azzerare, in ogni caso, il cronometro dello *spot*. Si è preferita un'azione simile perché di facile implementazione. Di maggior complessità sarebbe stata la soluzione più idonea, che prevede di riportare il contatore in uno stato congruo con la sequenza riordinata dei messaggi. Tuttavia, il parziale riallineamento proposto riconduce l'avamposto a una situazione accettabile. In primo luogo, viene fatta una copia dello *spot,* cosicché quello originale possa rimanere in uno stato consistente anche in fase di ripristino. A seguire,

un primo ciclo *for* svuota la coda, a partire dal messaggio più recente, finché non venga trovato un messaggio più vecchio di quello corrente. In tal caso, si entra nel blocco *if,* per poi uscire dal ciclo con un *break.* Si fa notare che, a ogni iterazione, lo stato dello *spot* locale viene aggiornato secondo il numero dei giocatori in zona, appena revisionato. Il meccanismo è un *rollback* che ripercorre lo storico, seguendo il flusso dei messaggi ricevuti. In aggiunta, vengono salvati i messaggi processati in una nuova coda temporanea. Il secondo ciclo, successivamente, ricomincia dal momento passato che è stato individuato come certo, e a questo integra i cambiamenti avvenuti a seguire. In tal maniera viene, anche, ricostruita la coda principale. Nell'elaborazione, però, si tiene ora conto dell'ultimo messaggio, prima disallineato. In conclusione, lo *spot* originale viene uniformato con la copia.

3.1.2 Lato client

Nella stessa maniera con cui si è anzitutto presentata la struttura delle classi lato server, si vuole adesso offrire al lettore una visuale della stessa lato client. Quindi, se da una parte ritroviamo un modello del dominio non lontano da quello del servitore, con classi quali "Player" e "Spot", dall'altra ce ne sono di nuove, come quella denominata "Bullet". Questa discrepanza può essere spiegata da un fattore pratico, ovvero la mancanza di necessità nel salvarsi centralmente i proiettili sparati. In tal modo, si evita di sovraccaricare in eccesso il server, in termini di memoria, e si lascia a ciascun giocatore la gestione di queste entità. Dato che ritengo questa la giusta linea di progettazione, c'è da dire che altre decisioni prese sono migliorabili in questo senso. Ma di ciò se ne discuterà nel capitolo conclusivo, in cui saranno trattati i possibili sviluppi futuri. Ad ogni modo, penso che i moduli più significativi siano quelli dei *controller,* tra cui il più importante "GameManager", che funge da burattinaio nel coordinamento delle altre entità. Questo modulo è di certo il nocciolo delle funzionalità offerte all'utente ed è quindi pensato come nodo stabile, assolutamente non modificabile ma estensibile. Di seguito, troviamo gli altri controllori "InputManager", "UIController", "NetworkManager" e "ProfileManager". Il primo tra questi si preoccupa di reagire agli input dell'utente, così da propagarli al "GameManager" e allo "UIController" mediante il meccanismo a eventi. Se da una parte questo modulo è l'unica dipendenza che possiede il "GameManager", dall'altra cambia assieme a esso e rimane indipendente dalla *view* che lo sfrutta. Infatti, lo "InputManager" offre all'esterno una serie di *callback* che qualsiasi *view* può richiamare all'occorrenza. Il secondo della serie si impegna ad aggiornare le interfacce grafiche, intese come pannelli, bottoni, testi, *slider* e altro del genere. Ciò vuol dire che è questo il modulo che cambia al variare delle interfacce utente. Proseguendo, risaltano due entità di maggiore interesse. In primis, il "NetworkManager" che si occupa della ricezione e dell'invio dei messaggi WebSocket. Di conseguenza, richiama le funzioni del "GameManager" all'arrivo dei messaggi, mentre li spedisce secondo sue direttive. Strettamente collegato al precedente, il "ProfileManager" traccia trasmissioni e ricezioni con la responsabilità di farne delle statistiche. Questi ultimi due moduli sono quelli che mutano in

base al protocollo di comunicazione utilizzato. Nel nostro scenario, infatti, sono queste le uniche due entità che vengono sostituite, senza stravolgere il resto della struttura. Tanto è che le dinamiche del gioco rimangono identiche in ambedue le opzioni di *networking*.

Dopo il resoconto effettuato, si vogliono mettere in luce alcuni aspetti importanti. Per prima cosa, si mostra come viene utilizzato il sistema di *scheduling* delle *callback,* offerto dal Cocos *engine*, al fine di inviare periodicamente posizione e rotazione del personaggio locale. La medesima API è stata utilizzata per i pacchetti di *profiling*.

```
private spawnPlayer(){
    const timestamps = [Date.now(), -1, -1, -1];
    let msg: Message = new Message(timestamps, MessageType.PlayerSpawned, JSON.stringify(GameManager.gameManager.localPlayer.toIdPosRotJSON()));
    this.socket.send(JSON.stringify(msg));
    this.schedule(() => this.changePosRot(), this._posRotMsgInterval, macro.REPEAT_FOREVER, this._posRotMsgInterval);
}
```

30. *snippet* di codice di "spawnPlayer" del "NetworkManager"

All'interno della classe "NetworkManager", il metodo privato "spawnPlayer" viene utilizzato per inviare un messaggio di tipo "PlayerSpawned" tramite le *JavaScript API For WebSocket,* come descritto nel capitolo 2.3.3. L'ultima riga, bensì, fa dell'altro. Infatti, la funzione "schedule" permette di pianificare l'invocazione di una *callback* in base a vari parametri. In questo caso, la *lambda expression*[22] contenente la chiamata a "changePosRot" viene eseguita ad intervalli regolari "_posRotMsgInterval", per un numero illimitato di volte, come suggerisce la costante "REPEAT_FOREVER", con un ritardo "_posRotMsgInterval" per la prima invocazione. Comunque, sono due i punti da precisare. Il primo riguarda l'apparente difetto di molteplici pianificazioni della stessa attività, ogni qualvolta che il "Player" locale si reincarna. Questa problematica rimane, per l'appunto, apparente, in quanto la funzione "schedule" non ha effetto se la *callback* passata è già registrata. Il secondo punto serve per ricordare al lettore la natura *single-threaded* dell'*engine*. Di fatti, queste *callback* registrate verranno eseguite in sequenza ad ogni *frame,* inserendosi in quel *main loop* del motore, precedentemente citato nel capitolo 2.3.1.

[22] Una *lambda expression* è una funzione anonima che in Java rappresenta un'istanza di un'interfaccia funzionale.

Una difficoltà incontrata, lato cliente, è stata quella di dover posticipare alcune reazioni ai messaggi ricevuti. Durante il cambio di scena, infatti, si è reso necessario attendere il suo completamento e assieme il termine della *callback* associata, prima di richiamare altre funzioni. Vediamo il perché con l'aiuto del codice.

```
toGameScene(){
    director.loadScene('Game', () => {
        if(!this._localPlayer || !this._spot){ director.loadScene('Home'); return;}

        this._localBox = director.getScene().getChildByPath('Canvas/World/LocalBox');
        this._remoteBox = director.getScene().getChildByPath('Canvas/World/RemoteBox');
        this._remoteBulletBox = director.getScene().getChildByPath('Canvas/World/RemoteBulletBox');
        this._camera = director.getScene().getChildByPath('Canvas/PlayerCamera').getComponent(Camera);
        this._redSpawner = director.getScene().getChildByPath('Canvas/World/RedSpawner');
        this._greenSpawner = director.getScene().getChildByPath('Canvas/World/GreenSpawner');
        this._spotSpawner = director.getScene().getChildByPath('Canvas/World/SpotSpawner');

        let spotComponent: Spot = instantiate(this.spotPrefab).getComponent(Spot);
        this._spot = this.cloneSpot(spotComponent, this._spot);
        this._spot.node.setParent(this._localBox);
        this._spot.node.position = this._spotSpawner.position;
        this._spot.onPlayerEnteredEvent(this.checkPlayerEnter, this);
        this._spot.onPlayerExitedEvent(this.checkPlayerExit, this);
        this._spot.onStatusChanged(() => this._eventTarget.emit(this.spotStatusChangedEvent, this._spot.status, this._spot.timer), this);

        this.spawnLocalPlayer();

        this._gameLoaded = true;
        this.callbacksAfterSpawning.forEach((call) => call());
        this.callbacksAfterSpawning = [];
    });
}
```

31. Snippet di codice di "toGameScene" del "GameManager"

Il metodo "toGameScene" viene invocato dal "NetworkManager" in due situazioni alternative. Se la partita è già in corso, con due o più giocatori all'interno, allora la funzione viene richiamata una volta ricevuto il messaggio "ListUpdate". Viceversa, il codice viene eseguito alla ricezione di un messaggio di "PlayerJoined", che indica la presenza di un secondo giocatore disponibile a iniziare il *match*. Nello *scope* della funzione, si nota come l'oggetto *singleton* "director" viene adoperato per caricare una nuova scena, dato che la gestione di queste ultime è a suo carico. Una volta che la scena è stata attivata, viene invocata la *callback* passata come secondo parametro. Adesso, ponendo da parte le dovute inizializzazioni dei campi del "GameManager", tra cui lo *spot* locale, e il successivo *spawn* del nodo *player*, si riscontra l'uso di un oggetto "callbacksAfterSpawning" di nostro primo interesse. In effetti, questo rappresenta un *array* che contiene quelle funzioni non invocabili prima del cambio di scena, e qui richiamate in sequenza. Se ne dà, di seguito, un esempio.

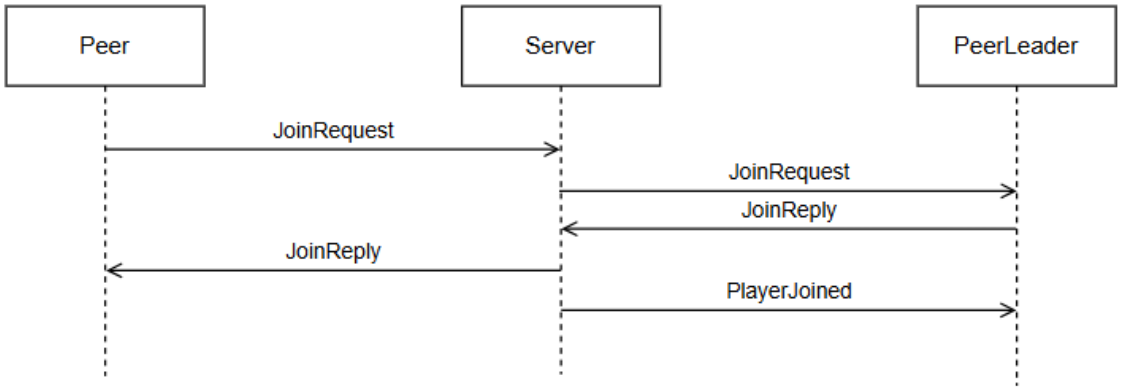
```
spawnRemotePlayer(value) {
    if(!this.gameLoaded) {this.callbacksAfterSpawning.push(() => this.spawnRemotePlayer(value)); return;}
    let playerComponent: Player = instantiate(this.playerPrefab).getComponent(Player);
    this._allPlayers.set(value._id, this.clonePlayer(playerComponent, this._allPlayers.get(value._id)));
    let remotePlayer: Player = this._allPlayers.get(value._id);
    remotePlayer.node.setParent(this._remoteBox);
    remotePlayer.node.getChildByPath('Character/Circle').getComponent(Sprite).color = new Color(remotePlayer.team === Team.Green ? this._greenRemoteColor : this._redRemoteColor);
    remotePlayer.status = PlayerStatus.Alive;
    remotePlayer.position = value._position;
    remotePlayer.rotation = value._rotation;
}
```

32. *snippet* di codice di "spawnRemotePlayer" del "GameManager"

La funzione “spawnRemotePlayer” si occupa di istanziare il *prefab* del giocatore come nodo nella scena. Per farlo necessita di uno specifico nodo riferito da “_remoteBox”, che funge da genitore per tutti i nodi funzionali a visualizzare i giocatori remoti, nella macchina locale. Il riferimento al nodo, però, rimane *null* fintantoché non venga inizializzato con lo *snippet* di codice mostrato in precedenza. Quindi, se la scena “Game” non è *loaded,* l’azione viene rimandata e salvata per il futuro nell’*array* menzionato prima, come mostra la prima riga della funzione. Da notare che si mantiene in memoria anche il parametro *value* così da riproporre esattamente la stessa invocazione.

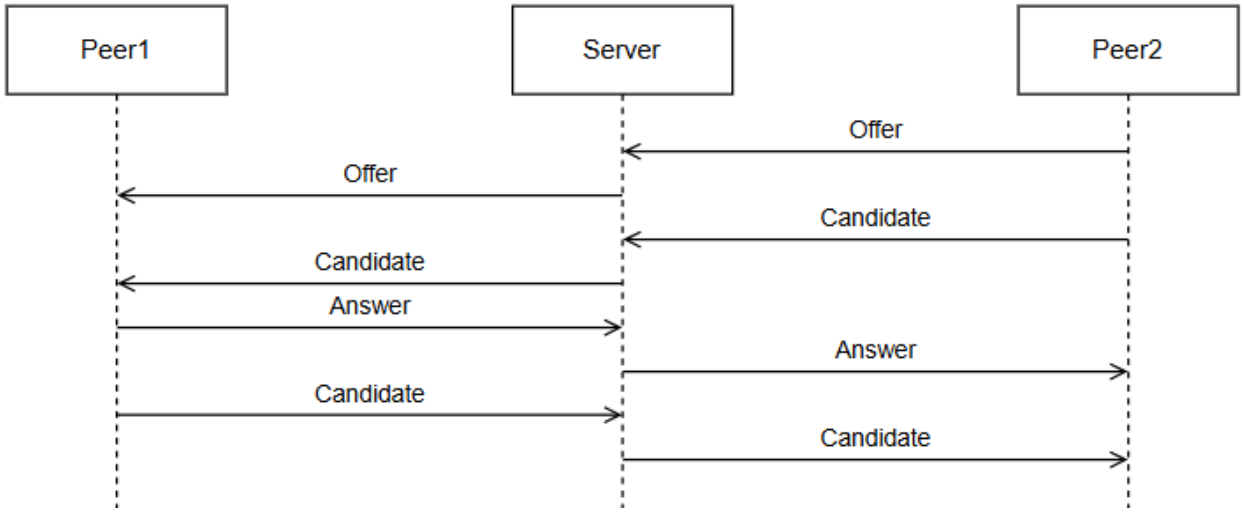
3.2 L'opzione peer-to-peer

Similmente a quanto illustrato nel sottocapitolo antecedente, si presenta una serie di diagrammi di sequenza che pone in rilievo alcune delle interazioni della seconda opzione di *networking*.



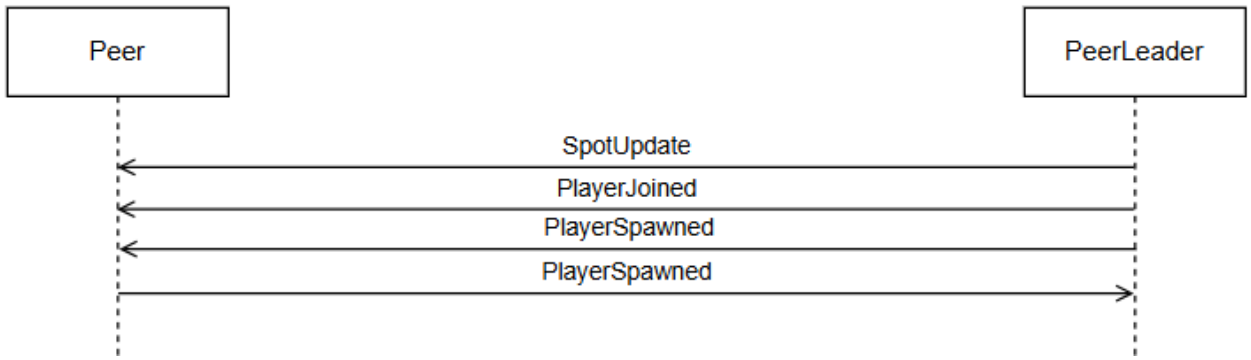
33. Diagramma di sequenza: *join* di un nuovo *peer*

In questo primo schema, sono presenti i messaggi scambiati fra tre entità diverse, in fase di partecipazione alla partita da parte di un nuovo *peer*. Dunque, come accade nella prima modalità, il giocatore che intende entrare in partita invia una "JoinRequest" al servente, con protocollo WebSocket. La differenza con la prima opzione risiede nel ruolo assunto dalla macchina server. Questa, infatti, si inserisce nella comunicazione tra *peer* al solo fine di farli conoscere e poco altro. Per tal ragione, il compito di indicizzare i giocatori è qui trasposto altrove, nel nodo individuato come *peer leader*. A esso viene rimbalzata, quindi, la richiesta di partecipazione, a cui risponde con un messaggio di "JoinReply", contenente *id* e team di appartenenza. Tale risposta viene instradata al destinatario dal server, che si preoccupa di avvisare ogni altro giocatore della comparsa del nuovo utente, mediante un "PlayerJoined".



34. Diagramma di sequenza: scambio *offer-answer* e candidati ICE

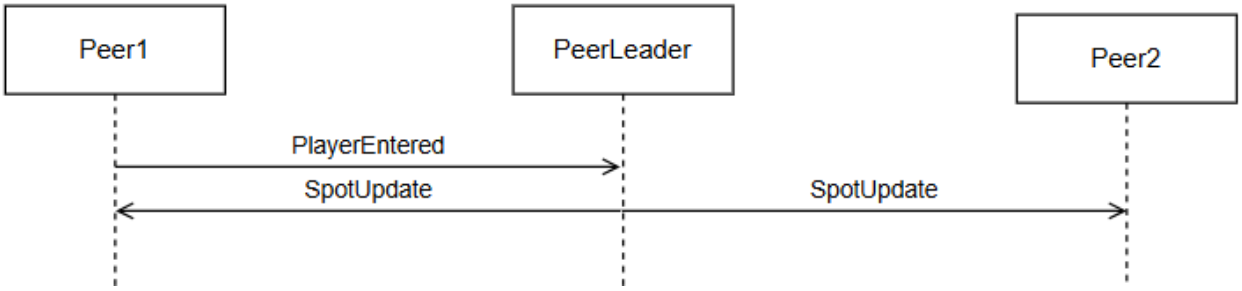
Questo secondo diagramma raffigura il processo di offerta e risposta proprio della fase di *signaling*. Assumendo il *peer* numero uno come quello appena connesso e il *peer* numero due come rappresentate di tutti gli altri già in partita, spieghiamo ora questo flusso di messaggi. Visto che il secondo *peer* è stato avvisato dell'evento, si appresta a preparare e trasmettere la sua "Offer" per una comunicazione SCTP *over* DTLS. Con questa, segnala anche la sua disponibilità allo scambio dei candidati ICE, secondo il protocollo Trickle ICE. Pertanto, appena possibile, si adopera a spedire quelli rinvenuti durante il processo di ricerca, tramite messaggi di tipo "Candidate". L'altro *endpoint* risponde con una "Answer", per accettare la proposta fatta. Anch'esso si impegna alla trasmissione dei suoi candidati. Nel complesso, si ricorda di nuovo la presenza costante del servente, che funge da intermediario tra i due.



35. Diagramma di sequenza: allineamento e *spawn*

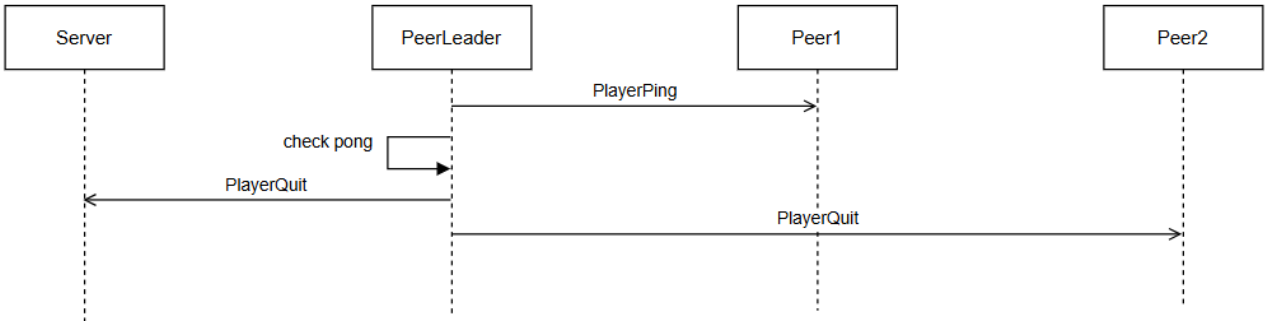
Nella figura di cui sopra, si conclude l'iter di preparazione per entrare in gioco. Perciò, all'interno di un'interazione ora WebRTC, il *peer leader*

comunica lo stato corrente dell'avamposto tramite il messaggio "SpotUpdate". Nonostante le tante somiglianze con l'opzione *client-server*, qui non compare un messaggio per aggiornare in blocco la lista dei partecipanti. Al contrario, ogni altro *peer* comunica la sua presenza in partita tramite un proprio "PlayerJoined", cosicché la lista del nuovo arrivato si possa formare progressivamente. Coloro i quali sono fisicamente nell'arena di gioco, e non temporaneamente deceduti, inviano anche un messaggio di "PlayerSpawned". Il medesimo è tramesso dal nuovo arrivato una volta che ne riceve almeno un altro prima e, al contempo, abbia già aggiornato il suo *spot* locale. Questo si traduce nella possibilità di procedere con lo *spawn* alla ricezione di un "PlayerJoined" o di uno "SpotUpdate".



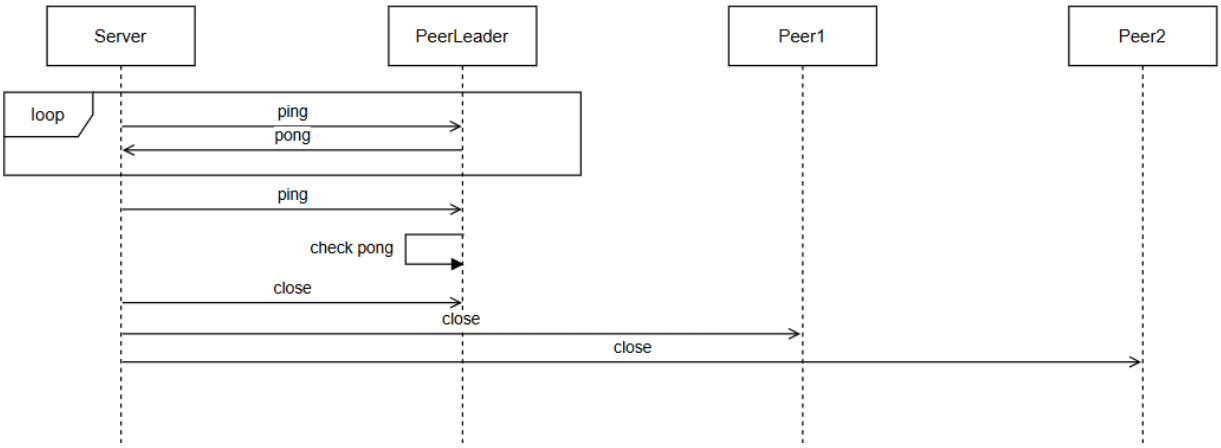
36. Diagramma di sequenza: assedio dell'avamposto

Con quest'altro schema si mostra al lettore lo scenario in cui il primo *peer* invade l'avamposto. Di conseguenza, ne viene informato il *peer leader* attraverso un messaggio di "PlayerEntered". Per la stessa ragione per cui il server verificava il *timestamp* di questo genere di messaggi, anche il *peer leader* assolve questo compito e si pone nel mezzo tra gli altri *peer*. Così facendo, questo supervisore ha modo di riordinare i "PlayerEntered" e i "PlayerExited" fuori posto. In ogni caso, il nuovo stato dello *spot* viene propagato con uno "SpotUpdate".



37. Diagramma di sequenza: *ping*, *pong* e abbandono del *peer*

Per rilevare disconnessioni, il *peer leader* si occupa di verificare la reattività dell'altro *endpoint* attraverso dei messaggi di tipo "PlayerPing". Anche se non mostrato in figura, normalmente gli altri *peer* rispondono con un "PlayerPong". Se così non fosse, però, allora il *peer leader* comunica agli altri *peer,* così come al server di *signaling,* la scomparsa del giocatore mediante un "PlayerQuit". In questo modo, si è voluto centralizzare il meccanismo di disconnessione di un *peer*.



38. Diagramma di sequenza: *ping, pong* e abbandono del *peer leader*

L'ultimo schema rappresenta lo scambio di *ping* e *pong* tra il server e il *peer leader*. In questo modo, il primo si assicura l'effettiva presenza del secondo, indispensabile per coordinare alcuni aspetti del gioco. Nel momento in cui, il servente non riceve risposta, si accinge a chiudere le connessioni WebSocket instaurate con tutti i giocatori. Questo fatto rappresenta per ciascun *peer* un segnale di fine partita a tutti gli effetti.

3.2.1 Lato server

Rimanendo un server WebSocket, la sua struttura non varia di molto rispetto a quella della modalità *client-server*. Di certo, il suo modello del dominio è molto più snello. Invero, l'unica classe che ne fa parte è quella denominata "Message", usata esattamente come in precedenza. Inoltre, permane un oggetto "SessionMessageQueue" come *controller* di utilità. Il fulcro del servente rimane l'*endpoint* WebSocket col nome "SignalingWebSocketController".

Per quanto riguarda l'oggetto "SessionMessageQueue" appena citato, questo è stato esteso con un nuovo costruttore, che accetta in ingresso il booleano "ping". Questo parametro serve per inibire o autorizzare il meccanismo di

ping-pong. In particolare, in questa modalità torna utile per impostarlo nel caso di connessione con il *peer leader* e per disabilitarlo per tutti le altre.

```
public SessionMessageQueue(Session session, boolean ping) {
    this.session = session;
    if(ping) startPingTask();
}
```

39. *Snippet* di codice del costruttore di "SessionMessageQueue"

Lasciando da parte questa piccolezza, si discute di seguito delle dinamiche per la scelta del *peer leader*. In merito, si sottolinea che questo supervisore viene stabilito in modo statico, senza possibilità di cambiare tale nodo nel corso della partita. La ragione sta ancora una volta nella semplicità dell'implementazione. Di fatti, l'utente che si connette per primo assume banalmente questo ruolo. Dunque, l'identificativo numero zero rappresenta sempre il *peer leader*. Ce ne dà conferma lo *snippet* di codice qui sotto.

```
public void onJoinRequest(Message msg, Session session){

    if(idSessions.size() == 0){
        idSessions.put(key:1, session);
        queues.computeIfAbsent(session, (s) -> new SessionMessageQueue(s, ping:true));
    }else{
        sessionsWithoutId.add(session);
        queues.computeIfAbsent(session, (s) -> new SessionMessageQueue(s, ping:false));
    }

    Session s = idSessions.get(key:1);
    joinRequest(msg.getValue(), s);
}
```

40. *Snippet* di codice di "onJoinRequest" del "SignalingWebSocketController"

Nello stesso *handler* di cui sopra, è osservabile un altro meccanismo implementato per mappare in ritardo gli oggetti "Session" con i propri identificativi. In effetti, si nota la coda "sessionsWithoutId" che raccoglie, con politica FIFO[23] (*First In First Out*), le sessioni dei nuovi arrivati. Queste potranno essere associate al proprio *id* solamente alla ricezione della "JoinReply", inviata dal *peer leader*.

[23] Una politica FIFO prevede che gli elementi della struttura dati siano inseriti in coda e rimossi dalla testa.

3.2.2 Lato peer

Relativamente alle classi e alle dipendenze tra queste, non c'è molto da aggiungere lato *peer* rispetto a quanto detto lato client per la modalità *client-server*. L'unica disparità la ritroviamo, come già predetto, nel modulo di *networking* e in quello a lui connesso di *profiling*. La classe che gestiste le comunicazioni viene quindi ribattezzata come "NetworkManagerPeerToPeer".

Si trattano ora alcune parti rilevanti lato *peer*. Innanzitutto, mettiamo il *focus* sull'utilizzo di un server STUN, per risolvere le problematiche di *address translation*. L'immagine sottostante adempie tale scopo.

```
if(msg._type === MessageType.Offer){
    const value = JSON.parse(msg._value);

    const remoteId = value._fromId;
    const localId = GameManager.gameManager.localPlayer.id;
    const remoteDescription = JSON.parse(value._value);
    const connection = new RTCPeerConnection({
        iceServers:[
            {urls: "stun:stun.l.google.com:19302"}
        ]
    });
    connection.ondatachannel = (event) => {
        const con = this._idConnections.get(remoteId);
        if(con){
            con.channel = event.channel;
            con.channel.onmessage = (event) => this.onPeerMessage(event);
        }
    }
    connection.onicecandidate = (event) => { if(event.candidate) this.sendCandidate(remoteId, localId, event.candidate); };
    this._idConnections.set(remoteId, new PeerConnection(connection, null));
    connection.setRemoteDescription(remoteDescription);
    connection.createAnswer().then((description) => {
        this.sendAnswer(remoteId,  localId, description);
        connection.setLocalDescription(description);
    });

    return;
}
```

41. *snippet* di codice dello "onMessage" del "NetworkManagerPeerToPeer"

In figura è mostrato il pezzo di codice dedicato alla "Offer" all'interno del più ampio *handler* "onMessage", pensato per gestire l'arrivo dei messaggi WebSocket. Si nota la costruzione dell'oggetto "RTCPeerConnection", con il passaggio di un oggetto con campo "iceServers". Questa proprietà è un *array* di elementi, ciascuno ricollegabile ad un server STUN o TURN, da utilizzare per il Trickle ICE. Nel nostro scenario, si è preferito adoperare solamente server STUN, così da proporre al lettore una vera modalità P2P. A seguire, si può osservare la sequenza di passi così come già descritta nel capitolo 2.3.3. In breve, le *callback* di reazione agli eventi importanti per il contesto vengono assegnate alle relative variabili. Il processo si conclude con la

creazione della risposta e con il suo invio. Si segnala l'uso del costrutto *then*[24], per gestire la "Promise" restituita da "createAnswer".

L'altro punto da analizzare riguarda il rilevamento della disconnessione. Da una parte il giocatore viene eliminato dalle strutture dati degli altri *peer* solamente alla ricezione del messaggio di "PlayerQuit", inviato dal *leader*. Tuttavia, è probabile che gli altri utenti si accorgano prima della mancanza dell'altro nodo. Spieghiamone il motivo con l'aiuto di due immagini.

```
private changePosRot(remoteId: number){
    const timestamps = [Date.now(), -1, -1, -1];
    let msg: Message = new Message(timestamps, MessageType.PosRotChanged, JSON.stringify(GameManager.gameManager.localPlayer.toIdPosRotJSON()));
    const channel = this._idConnections.get(remoteId).channel;

    if(!this.checkChannelAndQuit(channel, remoteId)) return;

    channel.send(JSON.stringify(msg));
}
```

42. Snippet di codice di "changePosRot" del "NetworkManagerPeerToPeer"

```
private checkChannelAndQuit(channel: RTCDataChannel, remoteId: number): boolean{
    if(!channel) return false;

    const close = channel.readyState === 'closed' || channel.readyState === 'closing';
    const open = channel.readyState === 'open';
    if(close){
        if(this._peerLeader == GameManager.gameManager.localPlayer.id){
            this.onChannelClose(remoteId);
        }
    }
    return open;
}
```

43. *Snippet* di codice di "checkChannelAndQuit" del "NetworkManagerPeerToPeer"

Nel momento in cui un giocatore deve comunicare agli altri un qualche tipo di messaggio, è possibile che uno dei *peer* si sia disconnesso. Quindi, prima di ogni trasmissione si verifica lo stato del *datachannel*. Qui si riporta il caso del messaggio di "PosRotChanged", che forse è il più esemplare, visto l'alta frequenza di invio. All'interno della prima funzione mostrata, ritroviamo l'invocazione della seconda dal nome "checkChannelAndQuit". Ebbene, sia nell'ipotesi in cui il canale non sia stato ancora creato in questo lato della connessione, sia nell'ipotesi in cui sia già chiuso o in chiusura, il messaggio non verrà trasmesso. Se chi spedisce il messaggio rappresenta il *leader*, allora questo si appresterà ad avvisare gli altri con un "PlayerQuit", richiamando il

[24] Il *then* rappresenta un metodo dell'oggetto Promise in JavaScript. Viene utilizzato per gestire il risultato della Promise in caso di successo. Al contrario, se ci sono errori, viene richiamato il codice passato tramite *catch*.

metodo “onChannelClose” e surclassando il meccanismo di *ping-pong* illustrato precedentemente.

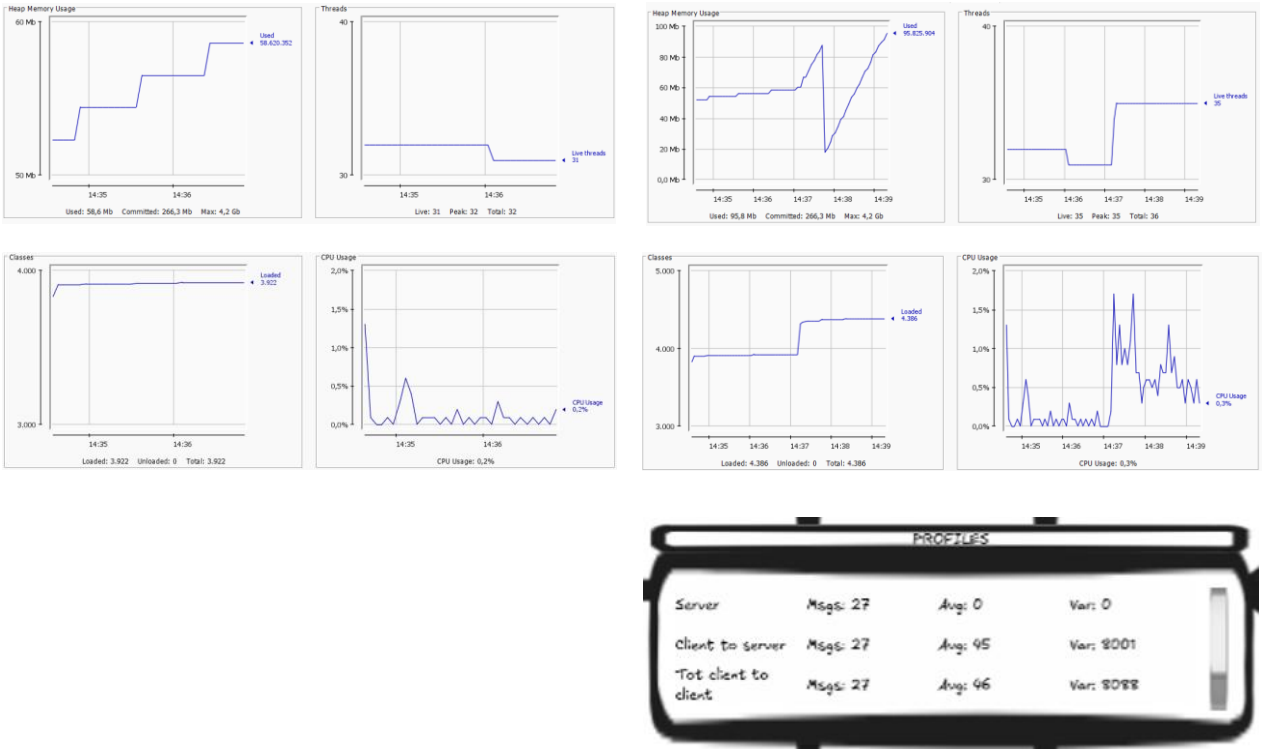
3.3 Le misure a confronto

Una volta implementate le due opzioni di *networking,* sono state effettuate delle misure per valutarne le prestazioni. Prima di mostrare i risultati del *profiling,* occorre spiegare al lettore l'ambiente dell'esperimento e gli strumenti adottati per le misurazioni. In primo luogo, si specifica che si sono utilizzati quattro *laptop,* differenti in qualità e *performance,* su cui distribuire il carico dei giocatori. Così facendo, sono riuscito a simulare fino a cento utenze per la prima modalità, fino a cinquanta per la seconda. Sulla macchina più potente è stato installato e configurato per l'esecuzione il *web server,* essenziale per entrambe le modalità. Sullo stesso *host* e sui rimanenti sono stati utilizzati *web client* variegati e ci si è connessi all'indirizzo del server raggiungibile sulla stessa LAN. Lato servitore, si è quindi impiegata la tecnologia di monitoraggio JMX (*Java Management Extensions)* offerta dalla stessa *suite* di Apache Tomcat. Assieme, si è sfruttato JConsole, incluso con il JDK, come strumento grafico. JMX fornisce statistiche in tempo reale sul consumo della *heap*, sul numero di *thread* attivi, sul numero di classi caricate e, infine, sull'uso della CPU (*Central Processing Unit)*. Dall'altro lato, invece, si è optato per la realizzazione di un modulo dedito al *profiling,* cioè la nuova classe "ProfileManager". Con essa, anche un nuovo oggetto del *model* pensato per salvare il numero di messaggi ricevuti, oltre a media e varianza delle latenze calcolate. Per finire, si sono progettati dei messaggi appositi di tipo "RTTRequest" e "RTTReply" con campi *timestamp,* inevitabili per il computo del RTT[25] (*Round Trip Time*).

3.3.1 L'opzione client-server

Si approfondiscono ora i risultati ottenuti nella modalità *client-server*, con l'aiuto dei grafici JMX e i valori numerici della *dashboard* nel gioco. Prima di proseguire con i commenti, si puntualizza che le misure lato cliente hanno le seguenti unità di misura: il numero dei messaggi è ovviamente adimensionale, mentre la media dei RTT è espressa in millisecondi e la varianza in millisecondi quadri.

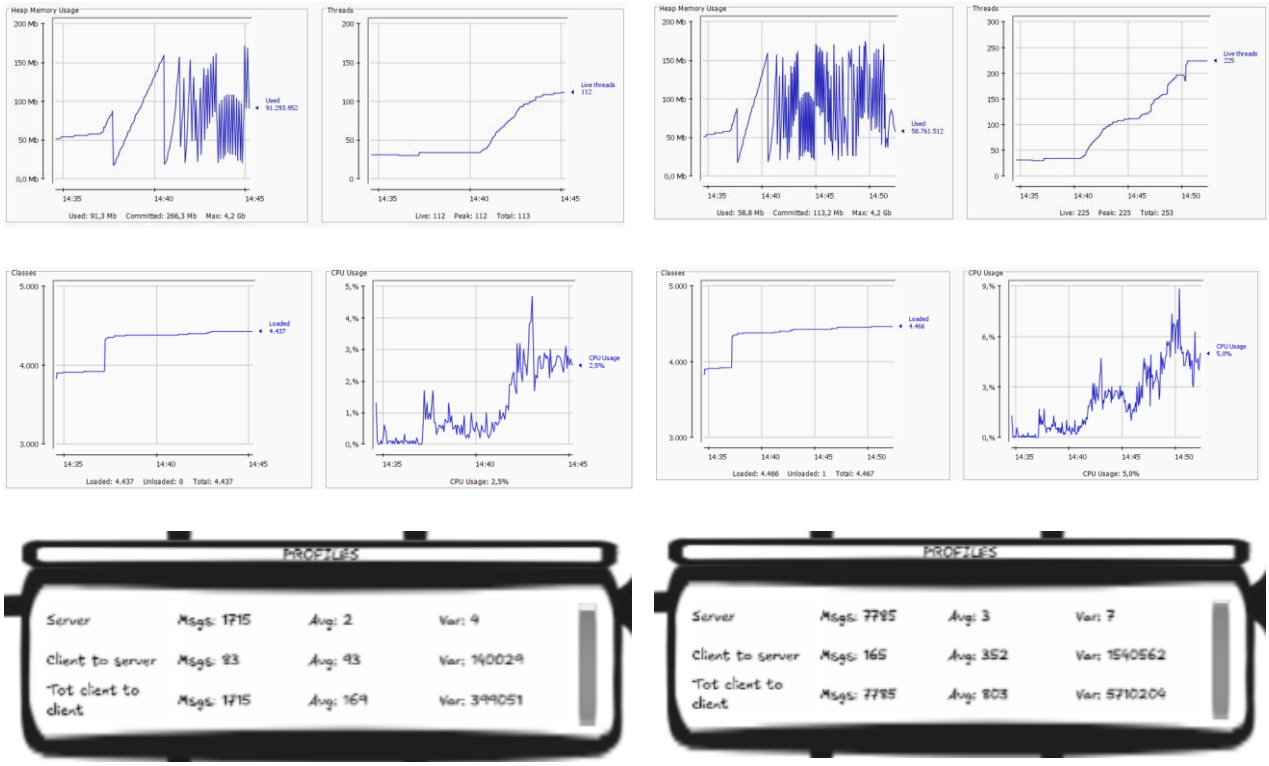
[25] Il RTT rappresenta il tempo impiegato da un pacchetto per raggiungere il nodo destinatario e per tornare indietro.



44. A sinistra: 0 utenti; a destra: 2 utenti

Dunque, come suggerisce la didascalia, i soli grafici di sinistra rappresentano la situazione del servente a inizio esecuzione, con zero utenti connessi. Si è voluto inserire questo primo scenario per osservare lo stato di partenza del server, così da calibrare su questo le letture dei prossimi risultati. Ad ogni modo, si fa notare il discreto *overhead* iniziale sull'uso della CPU, motivato dal processo di inizializzazione della JVM e dal lavoro compiuto per caricare le classi in memoria. Il *pool* di *thread* si stabilizza sul numero trentuno, tra cui una decina dedicata alle richieste secondo protocollo HTTP. Quest'ultimo dettaglio non è reperibile dalle immagini, ma proviene da ulteriori analisi svolte con JMX. Inoltre, la *heap* continua a crescere regolarmente finché la soglia massima per l'attivazione del *garbage collector* non sia raggiunta. Nella parte destra sono graficate le statistiche da inizio partita fino a qualche momento dopo la comparsa di due giocatori. Considerando anche gli attimi a venire, è possibile osservare i valori pressoché a regime dopo un primo transitorio. Quindi, si può individuare il momento della connessione dei due, associandolo al picco nel numero dei *thread*. Disponendo del codice, è possibile dare un volto ai nuovi quattro *thread*. Il primo tra questi è il processo di *dispatching* utilizzato da Tomcat per instradare i messaggi WebSocket verso il giusto *endpoint*. Un secondo rispecchia il timer demone per l'assedio dello *spot*. Infine, gli ultimi due corrispondono ai timer

predisposti per il meccanismo di *ping-pong,* uno per sessione. Il *thread* che esegue lo “OnOpen” non va conteggiato, perché ripescato tra uno di quelli già presenti nel *pool,* che ancora non è stato associato ad alcuna sessione attiva. Tra l’altro, questi stessi processi sono adoperati anche in caso di richieste HTTP, oltre alla gestione dei messaggi WebSocket. Ad ogni modo, se in numero insufficiente, vengono aggiunti nuovi *thread* nel pool. Sulla stessa coordinata temporale, distinguiamo dei picchi tanto nell’uso della CPU quanto nel numero di classi caricate. Se, da una parte, la prima percentuale si riassesta su valori irrisori, la seconda ovviamente non si smuove. Anche la *heap* ne risente e di conseguenza agisce la *garbage collection*. Per quel che riguarda i parametri calcolati su uno dei clienti, diverso da quello locale alla macchina server, se ne spiegano ora i valori: nella prima riga, si trovano media e varianza del tempo di elaborazione sul servitore dei messaggi di test; a seguire, i numeri del RTT *client-server*; infine, il totale dei RTT per il tragitto andata e ritorno tra il client locale e ciascun altro client.



45. A sinistra: 50 utenti; a destra: 100 utenti

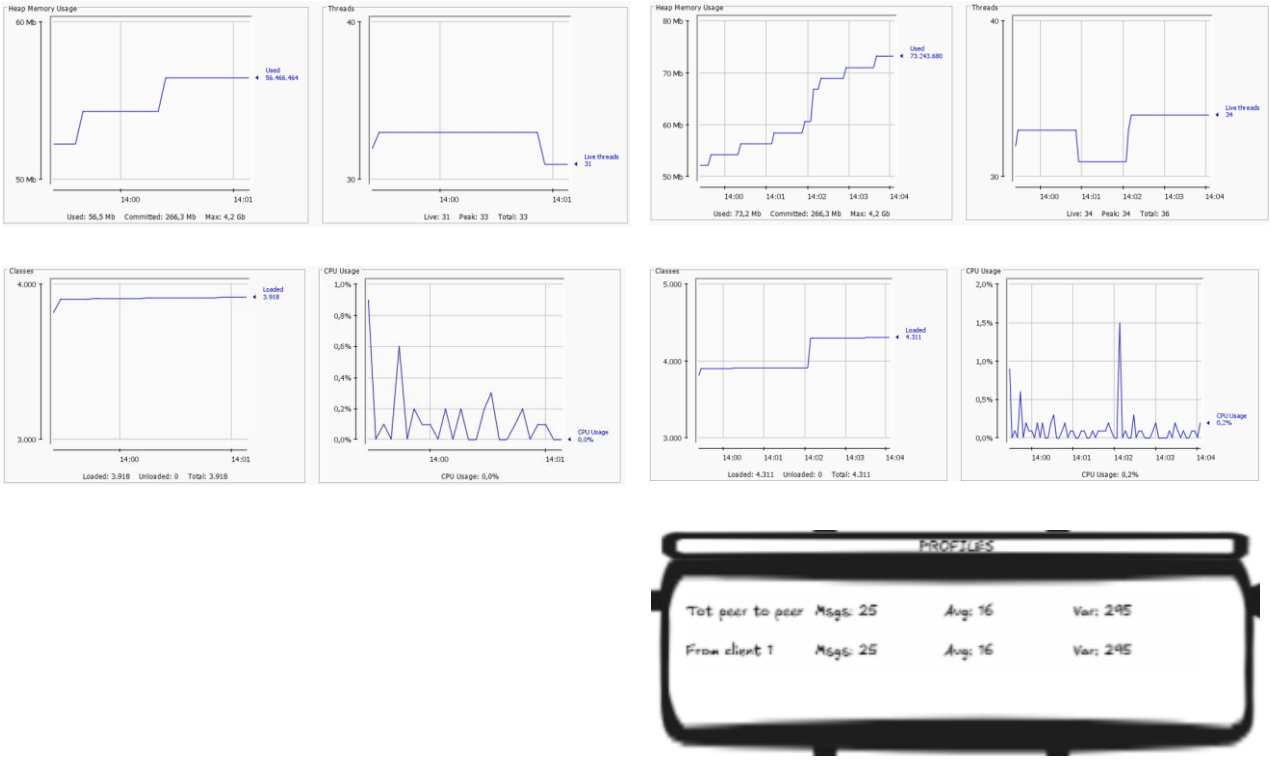
In questo secondo insieme, ritroviamo i tracciati con cinquanta utenti sulla sinistra, e con cento utenti sulla destra. Chiaramente, ogni variabile è in crescendo, ad eccezion fatta dell’uso della memoria, che viene ripetutamente

rinormalizzata dalla *tracing garbage collection*[26]. In particolare, si pongono in evidenza i picchi al nove percento della *CPU usage*, che poi rientra nell'intorno del cinque percento. Anche le statistiche lato cliente hanno tutti i parametri in aumento, ma direi non critici. Si mette in luce, specialmente, il RTT *client-to-client* che subisce un forte incremento medio. Quest'ultimo punto verrà sottolineato ancora nelle prossime sezioni.

[26] La *garbage collection* di tipo *tracing* consiste nello stabilire gli oggetti ancora vivi, cioè raggiungibili a partire dalle radici, per poi scartare quelli non marcati come tali, cioè *garbage*. Spesso, il componente che se ne occupa prevede anche un processo di compattazione della memoria successivo. Un'alternativa obsoleta è il *reference counting* che presenta un contatore di riferimenti per oggetto. Il contatore viene incrementato alla creazione di un riferimento che punta a quell'oggetto, mentre viene decrementato se accade il viceversa. Quando il contatore arriva a zero, allora l'oggetto può essere deallocato. Questa soluzione presenta, però, *memory leak* in caso di riferimenti ciclici.

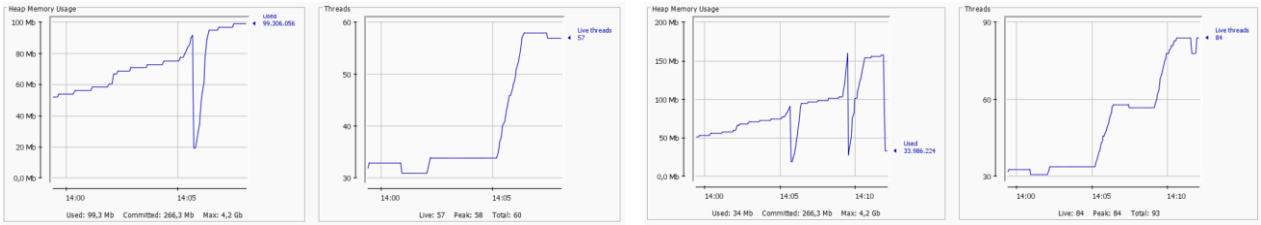
3.3.2 L'opzione peer-to-peer

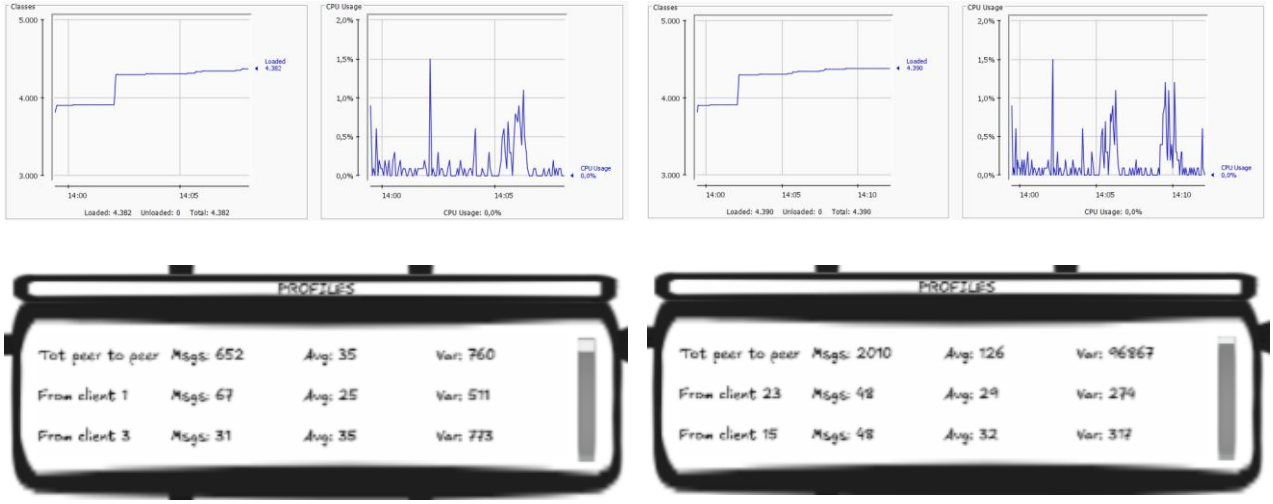
Alla stessa maniera e con uguali precisazioni, ci inoltriamo nelle misure fatte in modalità *peer-to-peer*.



46. A sinistra: 0 utenti; a destra: 2 utenti

Senza commentare con particolare cura ogni singolo riquadro, per non ripetere quanto già detto nel sottocapitolo precedente, si fanno notare le iniziali somiglianze con l'altra modalità. Tuttavia, è già possibile osservare una riduzione nel carico della CPU con due soli giocatori. Sarà più evidente nei grafici seguenti. Oltre a ciò, la media dei RTT, del tragitto con andata e ritorno tra il *peer* locale e i *peer* remoti, si rivela più bassa rispetto alla controparte *client-server*. Si dettaglia che la dicitura "From client x" tratta ugualmente di RTT, bensì specifico per il singolo cliente "x".





47. A sinistra: 25 utenti; a destra: 50 utenti

Con venticinque e cinquanta utenze, anche qui buona parte dei parametri crescono. Non è, però, il caso dell'uso della CPU lato server. Infatti, nonostante i picchi in fase di *signaling*, torna ad assestarsi su valori prossimi allo zero percentuale. Per quanto riguarda il numero dei *thread*, ci si aspetterebbe qui una crescita più moderata di questi, dato che vengono usati solo nel breve periodo di *signaling* e potrebbero essere efficacemente riutilizzati. Sarebbe così, se non fosse per la politica predefinita di Tomcat, per cui un *thread* è completamente dedicato ad una sola sessione specifica, senza possibilità di condividerlo. Lato cliente, notiamo di nuovo un forte balzo in avanti nella media totale dei RTT, come avveniva per l'altra opzione.

3.3.3 Il bilancio

Consapevoli delle statistiche ottenute, per quanto di natura didattica, è adesso possibile tirare le fila della sperimentazione. Innanzitutto, è lapalissiano come la soluzione P2P riduca drasticamente l'*overhead* sul server. Dunque, sviluppatori o *software house* che non dovessero disporre di molte risorse per i loro server, potrebbero ben volentieri considerare questa seconda opportunità. Vero che, oltretutto, le latenze tra i *peer* appaiono ridotte, in quanto i messaggi non devono passare per il server di mezzo. Tuttavia, la prima soluzione chiede meno sforzi ai *browser* dei clienti, che dovrebbero mantenere attiva una sola connessione, in condizioni normali. Nella nostra sperimentazione, però, questa circostanza non si è affatto verificata, a causa della penuria di macchine. Per tale motivo, il ritardo nei tempi di risposta si è fatto sentire tanto nella seconda opzione quanto nella prima. Probabile che questo fosse dovuto specialmente all'allungarsi dei tempi di attesa nei nodi clienti, e con peso minore all'allungarsi di quelli del nodo servente, proprio per il fatto che tale crescita è stata rivelata in ambedue le modalità, che condividono, appunto, le stesse estremità. Ad ogni modo, il quantitativo comunque maggiore di connessioni, per nodo cliente, ha reso impossibile per i *browser* spingersi oltre le cinquanta utenze, mentre in modalità *client-server* è stato possibile almeno un raddoppio di queste.

Conclusioni

Si è giunti al termine di questo elaborato e, dunque, ne si vuole anzitutto proporre un rapido sommario. La tesi ha primariamente introdotto il lettore all'argomento, specificandone gli obiettivi e le ragioni connesse. Nel primo capitolo, poi, sono state illustrate le opzioni di *networking* del videogioco, assieme ai protocolli di rete e alle tecnologie utilizzate. Nel farlo, però, si è preferito proporre diverse alternative a ciò che è stato effettivamente adoperato. In questo modo, è stato possibile, prima, offrire al lettore un vasto panorama e, solo poi, motivare le scelte tecnologiche fatte tra le tante sfruttabili. Nel secondo capitolo, ci si è maggiormente concentrati sui mezzi stabiliti. Perciò, sono stati discussi gli *standard* protocollari, le API correlate e gli ambienti di esecuzione, tanto sul servitore quanto sulla macchina dell'utente. Solamente nel terzo capitolo, comunque sufficientemente articolato, si è voluto raccontare del progetto e della sua implementazione. Diagrammi e *snippet* di codice ne hanno coronato la descrizione. A proposito del progetto, con esso hanno preso forma le iniziali intenzioni dell'autore, concretizzandosi in due prototipi funzionanti del videogioco, atti specialmente a dimostrare la fattibilità di quanto proposto e a saggiarne le prestazioni. Il contenuto è quindi fruibile da browser come stabilito, una volta recuperato all'indirizzo del *server* per cui è stato eseguito il *deployment*. Si è visto come i nodi utenti e il *server* comunicano, che sia per il solo *signaling* oppure per l'intera conversazione, mediante protocollo WebSocket. D'altra parte, WebRTC è il protocollo scelto nei canali tra *peer*. Con le tecnologie esposte nel capitolo secondo, quella di stampo JavaScript nel *frontend* e quella Java nel *backend*, si è effettivamente implementato il progetto. In fondo, l'ultimissima sezione del capitolo ha riportato l'attività sperimentale fatta. Di questa, sono stati analizzati i risultati delle misurazioni svolte e, nel bilancio di coda, ne è stata tratta una conclusione. In breve, si è appreso come l'architettura *client-server* richieda al fornitore del servizio una macchina potente da manutenere, così da ridurre notevolmente il carico sui nodi clienti. Al contrario, quella *peer-to-peer* aggrava i nodi utenti, che gestiscono a proprio conto le connessioni con i *peer* remoti. A parte la pecca, però, questa caratteristica diminuisce le latenze nello scambio dei messaggi

rispetto alla soluzione alternativa, dato l'assenza della tappa di passaggio aggiuntiva.

Per finire, si cita un repertorio di migliorie e possibili sviluppi futuri dell'opera fatta. In primo luogo, sarebbe opportuno semplificare il modello del dominio del servitore, eliminando una serie di classi e variabili non strettamente necessarie per questo nodo. Lo stesso può esser detto per il nodo cliente, che ha ben altre responsabilità. In breve, ottimizzare la struttura e partizionare il dominio in base alle esigenze dei due, senza ricopiarlo interamente. Inoltre, in fase di ripristino dello stato dell'avamposto, riportare anche il timer al giusto valore, nonostante le difficoltà. Nella modalità P2P, decentralizzare maggiormente la scoperta dell'abbandono da parte di un giocatore, senza chiamare in causa il *peer leader*. In più, rendere quest'ultimo dinamico, nel senso che possa essere variato anche durante la partita. In tal modo, si evita un singolo punto di fallimento. In generale, supportare molteplici partite in simultanea, utilizzando un nuovo identificatore. Quindi, integrare le interfacce d'ingresso tipiche dei dispositivi mobili. In ultimo, aggiungere un *nickname* per giocatore e decorare la grafica con effetti speciali.

Sitografia

- Alakkadshaw (2023). Google STUN server List. DEV Community. Ultimo accesso: 26 agosto 2025. https://dev.to/alakkadshaw/google-stun-server-list-21n4
- Alvestrand, H. (2021). Overview: Real-Time Protocols for Browser-Based Applications. RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc8825
- Amazon Web Services (s.d.). Che cos'è MQTT? AWS. Ultimo accesso: 26 agosto 2025. https://aws.amazon.com/it/what-is/mqtt/
- Apache Software Foundation (s.d.). Apache Tomcat. Apache Tomcat. Ultimo accesso: 26 agosto 2025. https://tomcat.apache.org/
- Apache Software Foundation (s.d.). Apache Tomcat 9 – Introduction. Apache Tomcat. Ultimo accesso: 26 agosto 2025. https://tomcat.apache.org/tomcat-9.0-doc/introduction.html
- Apache Software Foundation (s.d.). Apache Tomcat 9 – WebSocket How-To. Apache Tomcat. Ultimo accesso: 26 agosto 2025. https://tomcat.apache.org/tomcat-9.0-doc/web-socket-howto.html
- Apache Software Foundation (s.d.). Monitoring and Managing Tomcat. Apache Tomcat. Ultimo accesso: 26 agosto 2025. https://tomcat.apache.org/tomcat-9.0-doc/monitoring.html
- Baeldung (2023). A Guide to the Java API for WebSocket. Baeldung. Ultimo accesso: 26 agosto 2025. https://www.baeldung.com/java-websockets
- Cocos (s.d.). Lifecycle Callbacks. Cocos Creator Manual. Ultimo accesso: 26 agosto 2025. https://docs.cocos.com/creator/3.8/manual/en/scripting/life-cycle-callbacks.html
- DeepWiki (2023). Game Loop and Director. DeepWiki. Ultimo accesso: 26 agosto 2025. https://deepwiki.com/cocos/cocos-engine/2.2-game-loop-and-director
- Fette, I. & Melnikov, A. (2011). The WebSocket Protocol. RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc6455.html
- FreeCodeCamp (2023). JVM Tutorial – Java Virtual Machine Architecture Explained for Beginners. FreeCodeCamp. Ultimo accesso: 26 agosto 2025. https://www.freecodecamp.org/news/jvm-tutorial-java-virtual-machine-architecture-explained-for-beginners/
- GeeksforGeeks (2025). How JVM Works – JVM Architecture. GeeksforGeeks. Ultimo accesso: 26 agosto 2025. https://www.geeksforgeeks.org/java/how-jvm-works-jvm-architecture/
- GeeksforGeeks (2025). Interactive Connectivity Establishment (ICE). GeeksforGeeks. Ultimo accesso: 26 agosto 2025. https://www.geeksforgeeks.org/computer-networks/interactive-connectivity-establishment-ice/
- GeeksforGeeks (2025). Stream Control Transmission Protocol (SCTP). GeeksforGeeks. Ultimo accesso: 26 agosto 2025. https://www.geeksforgeeks.org/computer-networks/stream-control-transmission-protocol/
- GeeksforGeeks (2025). Structured and Unstructured Peer-to-Peer Systems. GeeksforGeeks. Ultimo accesso: 26 agosto 2025. https://www.geeksforgeeks.org/system-design/structured-and-unstructured-peer-to-peer-systems/#overlay-networks
- GeeksforGeeks (2025). What is Java Enterprise Edition (Java EE)? GeeksforGeeks. Ultimo accesso: 26 agosto 2025. https://www.geeksforgeeks.org/java/java-enterprise-edition/

- GeeksforGeeks (2025). What is Transmission Control Protocol (TCP)? GeeksforGeeks. Ultimo accesso: 26 agosto 2025. https://www.geeksforgeeks.org/computer-networks/what-is-transmission-control-protocol-tcp/
- GitHub (s.d.). cocos/cocos-engine. GitHub. Ultimo accesso: 26 agosto 2025. https://github.com/cocos/cocos-engine
- InnoGames (s.d.). Forge of Empires – Gioco di strategia online gratuito. Forge of Empires. Ultimo accesso: 26 agosto 2025. https://it-play.forgeofempires.com/
- Ivov, E., Uberti, J. & Saint-Andre, P. (2021). Trickle ICE: Incremental Provisioning of Candidates for the ICE Protocol. RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc8838
- Jakarta EE (s.d.). Jakarta WebSocket. Jakarta EE Tutorial. Ultimo accesso: 26 agosto 2025. https://jakarta.ee/learn/docs/jakartaee-tutorial/current/web/websocket/websocket.html
- JavaScript.info (s.d.). Long polling. JavaScript.info. Ultimo accesso: 26 agosto 2025. https://javascript.info/long-polling
- Jesup, R., Loreto, S. & Tüxen, M. (2021). WebRTC Data Channel Establishment Protocol. RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc8832
- Jesup, R., Loreto, S. & Tüxen, M. (2021). WebRTC Data Channels. RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc8831.html
- Keranen, A., Holmberg, C. & Rosenberg, J. (2018). Interactive Connectivity Establishment (ICE): A Protocol for NAT Traversal. RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc8445
- Khronos Group (s.d.). WebGL™ Overview. Khronos Group. Ultimo accesso: 26 agosto 2025. https://www.khronos.org/webgl/
- Localhorse.net (2025). Differences Between Javax and Jakarta. Localhorse.net. Ultimo accesso: 26 agosto 2025. https://localhorse.net/article/differences-between-javax-and-jakarta
- MDN Web Docs (2025). A simple RTCDataChannel sample. Mozilla Developer Network. Ultimo accesso: 26 agosto 2025. https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Simple_RTCDataChannel_sample
- MDN Web Docs (2025). HTTP: Hypertext Transfer Protocol. Mozilla Developer Network. Ultimo accesso: 26 agosto 2025. https://developer.mozilla.org/en-US/docs/Web/HTTP
- MDN Web Docs (2025). RTCPeerConnection. Mozilla Developer Network. Ultimo accesso: 26 agosto 2025. https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection
- MDN Web Docs (2025). The WebSocket API (WebSockets). Mozilla Developer Network. Ultimo accesso: 26 agosto 2025. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- MDN Web Docs (2025). WebGL API. Mozilla Developer Network. Ultimo accesso: 26 agosto 2025. https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API
- MDN Web Docs (2025). WebRTC API. Mozilla Developer Network. Ultimo accesso: 26 agosto 2025. https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
- Mo4tech (2023). Cocos Creator source code interpretation: engine start and main loop. Moment For Technology. Ultimo accesso: 26 agosto 2025. https://www.mo4tech.com/cocos-creator-source-code-interpretation-engine-start-and-main-loop.html
- Mohamed-Ali (2023). Server-Sent Events: A Comprehensive Guide. Medium. Ultimo accesso: 26 agosto 2025. https://medium.com/@moali314/server-sent-events-a-comprehensive-guide-e4b15d147576

- Nadun Kumarasinghe (2023). Building with Tomcat: Guide to Java Servlet Containers. Medium. Ultimo accesso: 26 agosto 2025. https://medium.com/@n3d/building-with-tomcat-guide-to-java-servlet-containers-0097cee9d4da
- Node.js (s.d.). Node.js — Run JavaScript Everywhere. Node.js. Ultimo accesso: 26 agosto 2025. https://nodejs.org/en
- Oracle (s.d.). Java. Oracle. Ultimo accesso: 26 agosto 2025. https://www.java.com/it/
- Oracle (s.d.). Java API for WebSocket 1.1 Maintenance Release. Oracle. Ultimo accesso: 26 agosto 2025. https://download.oracle.com/otndocs/jcp/websocket-1_1-mrel-eval-spec/index.html
- Oracle (s.d.). WebSocket (Java SE 11 & JDK 11). Oracle. Ultimo accesso: 26 agosto 2025. https://docs.oracle.com/en/java/javase/11/docs/api/java.net.http/java/net/http/WebSocket.html
- PixiJS (s.d.). PixiJS – The HTML5 Creation Engine. PixiJS. Ultimo accesso: 26 agosto 2025. https://pixijs.com/
- Python Package Index (s.d.). websockets. PyPI. Ultimo accesso: 26 agosto 2025. https://pypi.org/project/websockets/
- Python Software Foundation (s.d.). Welcome to Python.org. Python.org. Ultimo accesso: 26 agosto 2025. https://www.python.org/
- Redazione IONOS (2023). Cos'è il modello client-server? Pro e contro. IONOS Digital Guide. Ultimo accesso: 26 agosto 2025. https://www.ionos.it/digitalguide/server/know-how/modello-client-server/
- Redazione Managed Server (2025). Server TURN / STUN: cos'è ed a cosa serve? Managed Server. Ultimo accesso: 26 agosto 2025. https://managedserver.it/server-turn-stun-cose-ed-a-cosa-serve/
- Redazione (2023). Apache Tomcat. Come monitorarlo. ManageEngine Blog Italia. Ultimo accesso: 26 agosto 2025. https://blog.manageengine.it/apache-tomcat/
- RocketMe Up Networking (2023). Understanding VoIP Protocols — SIP, RTP, and More. Medium. Ultimo accesso: 26 agosto 2025. https://medium.com/@RocketMeUpNetworking/understanding-voip-protocols-sip-rtp-and-more-e1cc54e9f0af
- Socket.IO (s.d.). Socket.IO. Socket.IO. Ultimo accesso: 26 agosto 2025. https://socket.io/
- Stewart, R. (2007). Stream Control Transmission Protocol. RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc4960
- TypeScript (s.d.). TypeScript: JavaScript With Syntax For Types. TypeScript. Ultimo accesso: 26 agosto 2025. https://www.typescriptlang.org/
- Uberti, J., Jennings, C. & Rescorla, E. (2021). JavaScript Session Establishment Protocol (JSEP). RFC Editor. Ultimo accesso: 26 agosto 2025. https://www.rfc-editor.org/rfc/rfc8829.html
- Unity Technologies (s.d.). Unity Real-Time Development Platform. Unity. Ultimo accesso: 26 agosto 2025. https://unity.com/
- UniverseIT (2025). Event-driven architecture: cos'è l'architettura guidata dagli eventi. UniverseIT Blog. Ultimo accesso: 26 agosto 2025. https://universeit.blog/event-driven-architecture/
- V8 (s.d.). V8 JavaScript engine. V8.dev. Ultimo accesso: 26 agosto 2025. https://v8.dev/
- WebAssembly (s.d.). WebAssembly. WebAssembly.org. Ultimo accesso: 26 agosto 2025. https://webassembly.org/

- WebSocket.org (s.d.). WebSocket API. WebSocket.org. Ultimo accesso: 26 agosto 2025. https://websocket.org/reference/websocket-api
- Wikipedia (s.d.). Java virtual machine. Wikipedia. Ultimo accesso: 26 agosto 2025.https://en.wikipedia.org/wiki/Java_virtual_machine
- Wikipedia (s.d.). Peer-to-peer. Wikipedia. Ultimo accesso: 26 agosto 2025. https://it.m.wikipedia.org/wiki/Peer-to-peer
- Wikipedia (s.d.). Sistema client/server. Wikipedia. Ultimo accesso: 26 agosto 2025. https://it.m.wikipedia.org/wiki/Sistema_client/server
- Wikipedia (s.d.). Tim Berners-Lee. Wikipedia. Ultimo accesso: 26 agosto 2025. https://it.wikipedia.org/wiki/Tim_Berners-Lee
- Wikipedia (s.d.). WebRTC. Wikipedia. Ultimo accesso: 26 agosto 2025. https://it.wikipedia.org/wiki/WebRTC
- Yendis Entertainment (s.d.). Krunker.io – Free Online Multiplayer FPS Game. Krunker.io. Ultimo accesso: 26 agosto 2025. https://krunker.io/

Ringraziamenti

Vorrei qui ringraziare, con la più sincera gratitudine, il mio relatore, che prima di ogni altra cosa è stato mio professore, oltre ad avermi supportato tanto in questa prova finale, quanto da referente di tirocinio. Ringrazio anche la mia cara zia, che si è sempre resa disponibile per revisionare queste e tante altre pagine. Ringrazio i miei genitori, per avermi sostenuto fino qua emotivamente ed economicamente. Un grazie va, infine, alla mia compagna e ai miei amici, per i momenti spensierati e gli affetti scambiati.